RAG KNOWLEDGE SYSTEM

RAG 知识体系
融会贯通学习手册

检索增强生成 —— 从原理到工程落地的完整知识框架

内容来源：IMA 知识库「RAG」11 篇系列文章

覆盖板块：为什么需要 RAG · 完整链路 · 切片策略 · Embedding · 向量数据库 · 排障诊断 · 优化思路 · 长文档与代码检索 · Agentic RAG · 评估体系 · 面试框架

版本：2026 年 10 月整合版

> 阅读建议：先看「第 0 章 60 分钟速通」掌握 20 条核心结论，再按「为什么 → 怎么搭 → 怎么优化 → 怎么排障 → 怎么评估 → 怎么答」的主线顺序精读。面试前速刷附录 D 的 30 问速答卡。

目 录

共 20 章 · 116 个知识单元（点击可跳转）

写在前面：这份文档怎么用

- 一、全书只有一条主线
- 二、知识之间的四种逻辑关系
- 三、推荐学习路径

第 0 章 60 分钟速通：20 条核心结论

第 1 章 为什么需要 RAG：大模型的四大局限与 RAG 的应对

- 1.1 从一个真实的困境说起
- 1.2 大模型的四大内生局限
- 1.3 RAG 的应对机制
- 1.4 RAG 不是万能的：适用与不适用
- 1.5 与其他方案的关系：Prompt 塞文档 / 微调 / RAG
- 1.6 面试怎么问
- 1.7 本章常见误区
- 1.8 本章关键结论

第 2 章 RAG 完整链路：离线索引 + 在线检索生成

- 2.1 完整链路总览
- 2.2 离线阶段逐段拆解
- 2.3 在线阶段逐段拆解
- 2.4 三条核心认知（必须记住）
- 2.5 本章常见误区
- 2.6 面试怎么问
- 2.7 本章关键结论

第 3 章 切片策略 Chunking：决定检索质量的地基

- 3.1 用一个比喻建立直觉
- 3.2 四种主流切分策略对比
- 3.3 Overlap：被低估的关键参数
- 3.4 Chunk 过大、过小各有什么问题?
- 3.5 Chunk 大小经验值
- 3.6 实际项目里怎么选策略（决策框架）
- 3.7 面试怎么问

· 3.8 本章关键结论

第 4 章 Embedding：语义压缩、模型选型，与 Rerank 的区别

· 4.1 一个经典对话引入问题
· 4.2 Embedding 到底是什么
· 4.3 Embedding 不是生成文本，是在压缩语义
· 4.4 Embedding 模型怎么选（四个维度）
· 4.5 Embedding 和 Rerank 有什么区别（高频面试题）
· 4.6 中文场景和通用场景的区别
· 4.7 四大误区
· 4.8 面试怎么问
· 4.9 本章关键结论

第 5 章 向量数据库：ANN 索引原理与选型

· 5.1 从一个具体问题出发
· 5.2 为什么不能用 MySQL 做向量检索（高频面试题）
· 5.3 ANN：如何在速度和精度之间取得平衡
· 5.4 向量数据库还能做什么（三个工程能力）
· 5.5 主流向量数据库与选型建议
· 5.6 四大误区
· 5.7 面试怎么问
· 5.8 本章关键结论

第 6 章 排障诊断：检索侧五类 + 生成侧四类问题

· 6.1 总原则：先分流，再定位
· 6.2 检索侧五类问题
· 6.3 生成侧四类问题
· 6.4 如何系统地诊断 RAG 问题（四步法）
· 6.5 面试怎么问
· 6.6 本章关键结论

第 7 章 优化思路：Query / 检索 / 精排 / Context 四侧

· 7.1 优化要对症下药
· 7.2 Query 侧优化
· 7.3 检索侧优化
· 7.4 精排优化：Rerank
· 7.5 Context 侧优化
· 7.6 初学者应该如何选择优化方向（四条原则）

· 7.7 面试怎么问
· 7.8 本章关键结论

第 8 章 长文档与代码检索：结构、版本、权限

· 8.1 三个硬问题
· 8.2 简要回答（先记住这五条）
· 8.3 固定切片为什么在长文档里容易失效
· 8.4 代码为什么不能按字符数硬切
· 8.5 Parent-Child 怎么兼顾找得准和读得懂
· 8.6 为什么版本和权限必须进入索引
· 8.7 在线检索链路怎么跑（七步）
· 8.8 命中子块后，要不要总是返回整个父块?
· 8.9 项目里怎么落地和评估
· 8.10 知识拓展 Q&A
· 8.11 本章关键结论

第 9 章 Agentic RAG：从固定流水线到动态检索循环

· 9.1 问题的起点
· 9.2 RAG 的四阶段演进（递进关系）
· 9.3 Agentic RAG 到底要解决什么问题（五类）
· 9.4 核心原理：五个角色 + 一个循环
· 9.5 用一个例子走完完整过程
· 9.6 Agentic RAG 不等于多 Agent
· 9.7 它比传统 RAG 强，但也更难上线
· 9.8 什么场景值得用 Agentic RAG
· 9.9 面试里怎么解释 Agentic RAG
· 9.10 本章关键结论

第 10 章 评估体系：检索质量 + 生成质量 + 线上采用率

· 10.1 为什么 RAG 必须有评估体系
· 10.2 检索侧评估：四个核心指标
· 10.3 生成侧评估：RAGAS 四个核心维度
· 10.4 RAGAS 四个指标的关系
· 10.5 如何构建 RAG 评估集
· 10.6 离线评估 vs 在线评估
· 10.7 评估驱动的优化闭环（五步）
· 10.8 知识拓展 Q&A
· 10.9 本章关键结论

第 11 章 面试应答框架：高频问题与回答结构

- 11.1 面试官真正想听什么
- 11.2 五个落点（回答任何 RAG 问题的骨架）
- 11.3 面试官会沿着项目一直往下挖（追问阶梯）
- 11.4 通用回答框架（五步）
- 11.5 八个高频问题怎么答
- 11.6 面试里最容易暴露的四个问题
- 11.7 如果只做过 Demo，怎么答?
- 11.8 知识拓展 Q&A
- 11.9 本章关键结论

附录 A：核心公式速查

附录 B：指标体系速查表

附录 C：参数经验值速查

附录 D：高频面试题速答卡（30 问）

附录 E：常见误区总清单

附录 F：术语表

全书收束：一页记住 RAG

副标题：检索增强生成 —— 从原理到工程落地的完整知识框架

内容来源：IMA 知识库「RAG」11 篇系列文章（为什么需要 RAG / 完整链路 / 切片策略 / Embedding / 向量数据库 / 排障诊断 / 优化思路 / 长文档与代码检索 / Agentic RAG / 评估体系 / 面试框架）重新梳理、合并、串联而成。

使用目标：读完这一份，不必再去翻零散文章；脉络清晰，重点突出，可直接用于系统学习与面试准备。

写在前面：这份文档怎么用

一、全书只有一条主线

```
为什么需要 RAG  →  怎么搭（链路）  →  怎么优化  →  怎么排障  →  怎么评估  →  怎么答（面试）
   第 1 章          第 2~5 章         第 7 章      第 6 章      第 10 章      第 11 章
                                        ↑            ↑
                                   第 8、9 章（进阶形态）
```

一句话记忆：**先证明必要性，再拆解流水线，接着逐个环节打磨，出问题能定位，改完能量化，最后能讲清楚。**

二、知识之间的四种逻辑关系

理解 RAG 不是背知识点，而是理解知识点之间的四种关系：

关系	含义	典型例子
递进	后者解决前者解决不了的新问题	传统 RAG → Advanced RAG → 自适应 RAG → Agentic RAG
包含	大概念由若干小环节构成	完整链路 ⊃ 离线索引 + 在线检索生成
因果	前一个环节的质量决定后一个环节的上限	文档质量差 → 向量质量差 → 检索召回差 → 答案差
对比	两者解决不同层次的问题，不能互相替代	Embedding（粗筛）vs Rerank（精排）；混合检索（补召回）vs Rerank（提排序）

三、推荐学习路径

阶段	章节	目标	建议投入
① 建立认知	第 1 章	说清"为什么需要 RAG"，能判断适用边界	30 分钟
② 掌握主干	第 2 章	能完整复述一条 RAG 链路	40 分钟
③ 深入地基	第 3、4、5 章	切片 / Embedding / 向量库三大选型决策	120 分钟
④ 学会定位	第 6 章	答案不准时能先分流再定位	40 分钟
⑤ 学会改进	第 7、8、9 章	四侧优化手段 + 进阶形态	120 分钟
⑥ 学会证明	第 10 章	用指标体系量化证明方案有效	60 分钟
⑦ 学会表达	第 11 章	把项目讲成"发现问题—取舍—验证"的链路	60 分钟
⑧ 考前速刷	第 0 章 + 附录	20 条核心结论 + 速答卡	30 分钟

> 如果只有 1 小时：直接读**第 0 章速通清单**，再读**附录 D 高频面试题速答卡**。

第 0 章 60 分钟速通：20 条核心结论

> 这一章是全书骨架，任何一条展开都是后面某一章的浓缩。先记住它，后面都是加细节。

1. **RAG 不是模型能力的升级，而是模型的外置数据库**——让模型回答时能查自己的 cheat sheet。
2. 大模型有四个内生局限：**幻觉、私有知识盲区、知识时效性、无法溯源**。RAG 各有对应解法，但**只能缓解不能根治。**
3. RAG 的本质是**一条工程流水线**，不是一个算法。
4. 链路分两段：**离线侧负责"知识怎么进去"，在线侧负责"知识怎么找出来、怎么用起来"。**
5. **离线与在线必须严格一致**：Embedding 模型、清洗方式不一致，向量空间就不同，相似度计算彻底失效。
6. **检索的目标是精准，不是全面。**top-3 的高质量 chunk 常优于 top-20 的混杂结果。
7. **Chunking 是 RAG 最底层的地基**，模型再强也弥补不了检索质量的缺陷。
8. Chunk 的核心矛盾：**太小语义不完整，太大信噪比太低。**
9. 切片四策略：固定长度（原型用）、递归字符（最常用默认）、语义切分（最贵最准）、**结构感知（有结构时最优）**。
10. **Overlap 是防止边界信息被割裂**，推荐值为 chunk 大小的 10%~20%，不是越大越好。
11. **Embedding 不是生成文本，是把语义压缩进一个可计算的空间。**
12. **Embedding 与 Rerank 的区别**：前者双塔、独立编码、可离线预计算，负责粗筛；后者交叉编码器、联合建模、逐对推理，负责精排。
13. **换 Embedding 模型必须全量重建索引**，否则新旧向量不可比。
14. 向量数据库解决的是传统数据库**从未设计来解决的问题**：海量高维向量中的近似最近邻搜索。
15. **ANN 通过 HNSW / IVF 索引把检索从 O(N) 降到近似 O(log N)**，代价是"近似"——召回精度通常 95%+，对 RAG 足够。
16. **排障第一步永远是先分流**：手动看 top-K 里有没有正确答案 → 有是生成侧问题，没有是检索侧问题。
17. **优化的顺序：先建 baseline，再解决"能不能找到"，最后解决"找到了对不对"；每次只改一个变量。**
18. **父子块检索化解"检索用小块、返回用大块"的矛盾**；长文档与代码还要多守两条：**版本不能错、权限不能漏。**
19. **Agentic RAG 增加的不是一层 Agent 外壳，而是检索过程中的决策权、反馈和停止条件。**
20. **离线指标是方向盘，线上采用率是目的地。** 没有评估的优化，就是拿着锤子找钉子。

第 1 章 为什么需要 RAG：大模型的四大局限与 RAG 的应对

1.1 从一个真实的困境说起

假设公司有几千份内部文档：产品手册、操作规范、历史项目经验、客户合同……领导想做一个 AI 问答系统，员工随时可以问"我们的退款政策是什么"、"这个客户签了哪些条款"，系统直接给准确答案。

直接用大模型做，第一轮测试效果不错，但很快撞墙：

- 模型对公司内部文档**一无所知**，回答全靠"编"；
- 产品文档上周刚更新，模型说的还是**老版本**内容；
- 回答很流畅，但**无法判断真假、无法追溯来源**；
- 把文档加载进 Prompt，**token 成本高到无法接受。**

这不是模型能力不够，而是大模型作为一种技术，有几个**根本性的天花板。**RAG 就是为了突破这些天花板而诞生。

1.2 大模型的四大内生局限

局限	本质原因	危害
幻觉（Hallucination）	大模型是概率语言模型，生成每个 token 时做的是"预测下一个最可能的词"，而不是"查知识库"	不知道答案时不会说"我不知道"，而是生成一个"听起来最合理"的答案；语气坚定、格式工整，难以辨别真假。在企业场景里，它会**以可信的方式传播错误信息**
私有知识盲区	模型只在公开数据上训练，公司内部文档从未见过	全部塞进 Prompt 在文档量大时 token 成本与上下文长度都会爆；且**模型在极长上下文里注意力严重稀释**，对中间内容利用率极低
知识更新滞后	训练有截止日期，通常落后现实 6 个月到一年	价格变了、政策修订了、新竞品出现了，模型全不知道；重新训练的**成本和时间都不可接受**
无法追溯来源	生成过程不保留出处	说不出"这句话来自哪份文档第几页"。法律、医疗、金融等场景**没有来源引用的答案往往不可接受**

> **重点提醒**：这四个问题是理解 RAG 一切的出发点。后面每一章的设计决策，本质上都在服务这四个问题的某一个。

1.3 RAG 的应对机制

RAG 全称 **Retrieval-Augmented Generation（检索增强生成）**。核心思路非常直觉：

> **在让模型回答之前，先去知识库里检索出最相关的内容，把这些内容作为上下文一起喂给模型，让模型"看着资料"来回答。**

RAG 系统的基本构成：

```
用户 Query
   ↓
[检索器 Retriever] ↔ [外部知识库 / 向量库]
   ↓ 相关片段 + 来源
[生成器 (LLM)]
   ↓
有根据的答案（附来源、可核查）
```

四对四的应对关系：

大模型局限	RAG 的应对	效果边界
幻觉	检索到的原文片段作为上下文传入，模型被要求"根据以下资料回答"，有了事实锚点	**大幅降低但非归零**：模型仍可能在检索结果上做错误推理，或在检索质量差时"补充生成"
私有知识	知识库完全自建自维护，可包含任何内部文档；模型只需在检索片段上推理和组织语言	前提是文档被正确解析、切片、入库
知识更新	更新知识库不需重训模型，只需更新文档、重做向量化索引	知识新鲜度完全由知识库决定，与模型训练截止日期**解耦**
可追溯	每次检索都知道命中了哪份文档的哪个片段，可在答案里附引用来源	依赖元数据完整（来源、页码、时间）

1.4 RAG 不是万能的：适用与不适用

适合 RAG 的场景：

- **知识密集型问答**：企业内部知识库、产品手册问答、法律法规查询——事实准确性要求高、内容量大、需要来源追溯。
- **内容频繁更新的业务**：新闻摘要、竞品分析、政策解读——知识库持续接入实时数据，模型本身不用动。
- **长尾事实查询**："公司在上海的办公地址是什么""这个合同的甲方是谁"——答案藏在特定文档里，必须检索。

不适合 RAG 的场景：

- 创意生成类任务；
- 强推理任务（RAG 只提供事实，不提供推理能力）；
- 模型训练数据已良好覆盖的稳定领域——此时 RAG 只增加延迟和成本，没有实质收益。

1.5 与其他方案的关系：Prompt 塞文档 / 微调 / RAG

方案	适合解决	不适合解决
Prompt 塞文档	少量文档、临时性需求	文档量大时 token 成本与上下文长度爆炸
微调（Fine-tuning）	固定格式、固定语气、领域表达习惯	频繁变化的动态知识（成本高、更新慢）
RAG	**外部知识接入、动态知识、需要来源引用**	不能替代模型自身的推理能力；不是给所有模型问题贴创可贴

> 判断顺序：**先问是不是"外部知识接入"问题。**如果是格式或语气问题，先用 Prompt，必要时才考虑微调。

1.6 面试怎么问

Q：为什么有了大模型还需要 RAG?

参考思路：大模型有四大局限——幻觉、私有知识盲区、知识时效性、无法溯源。RAG 在回答前先检索外部知识库，把相关文档片段注入上下文，让模型"看资料回答"，从而有针对性地缓解以上问题。

Q：RAG 能解决幻觉问题吗?

参考思路：能显著缓解，但**无法根治。**有了检索到的事实作为锚点，模型乱编的概率大幅降低，且答案可附来源供核查。但模型仍可能在检索结果上做出错误推理，或在检索质量差时"补充生成"。**RAG 降低幻觉概率，不消灭幻觉根因。**

Q：什么场景下推荐用 RAG，什么场景下不用?

参考思路：适合知识密集、内容频繁更新、需要溯源的场景（企业知识库问答、法规查询）；不适合创意生成、强推理任务、以及模型训练数据已良好覆盖的稳定领域——在这些场景下 RAG 只增加延迟和成本。

1.7 本章常见误区

- ❌ "大模型有幻觉，所以所有场景都要上 RAG。" —— 太泛。大模型有幻觉，不代表所有场景都要建知识库。
- ❌ "RAG 能彻底消除幻觉。" —— 不能，它只是降低了**无依据生成**的概率。

1.8 本章关键结论

> RAG 是**大模型的外置数据库。**它不改变模型能力，只是在模型回答前把"该看的资料"放到它眼前。

第 2 章 RAG 完整链路：离线索引 + 在线检索生成

> 这一章是全书的**主干骨架**。后面每一章讲的都是这条链路上某一个环节的深入。

2.1 完整链路总览

整条链路分两段：**离线侧负责"把知识存进去"，在线侧负责"把相关知识找出来、组合出答案"。**

```
【离线：索引构建】
原始文档（PDF/Word/网页…）
   ↓ 文档加载 Document Loading
文档处理（清洗、解析格式）
   ↓
切片 Chunking
   ↓
向量化 Embedding
   ↓
向量库（向量索引）+ 元数据存储（来源、标题、时间…）

【在线：检索生成】
用户 Query
   ↓
Query 向量化（必须用同一个 Embedding 模型）
   ↓
检索器 Retriever（向量检索 / 混合检索）
   ↓
Rerank 精排（可选）
   ↓
上下文构建 Context（拼装 + 来源标注）
   ↓
生成器 LLM（Query + Context → 回答）
   ↓
有依据的答案 + 来源引用
```

2.2 离线阶段逐段拆解

① 原始文档（Document Loading）

知识来源多样：PDF 报告、Word 文档、网页、Markdown、数据库记录、邮件……不同格式需要不同解析方式。

> **关键认知**：这一步的质量直接决定整个系统的上限。如果原始文档本身是扫描件或排版混乱的 PDF，解析出的文本充满噪声，后续所有环节都会受损。**"垃圾进，垃圾出"在 RAG 里体现得非常明显。**

② 文档处理（清洗与预处理）

解析出的原始文本不能直接用，需要一轮清洗：去掉页眉页脚、无意义格式符号、重复内容；识别并保留文档标题结构；过滤表格乱码、图片占位符。

> 这一步看起来琐碎，但实际项目里，**文档预处理往往是工程量最大、最容易被低估的部分。**

③ 切片（Chunking）

清洗好的文档不能整篇塞进向量库，要切成更小的片段（chunk）。这是 RAG 里**设计决策最多**的环节，直接影响检索精准度。

为什么要切？ 一篇 20 页的文档，用户的问题可能只与其中某一段相关。整篇作为一个单元存储和检索，要么粒度太粗（命中了整篇但相关内容被淹没），要么上下文太长（放不进模型或注意力被稀释）。

切多大合适？ 没有通用答案，取决于文档类型、模型上下文窗口、业务问题的颗粒度。→ **详见第 3 章**

④ 向量化（Embedding）

每个 chunk 通过 Embedding 模型转成向量（高维浮点数数组），代表这段文字的"语义"。

> **铁律**：用户问题和文档 chunk 必须用**同一个 Embedding 模型**处理，两者才处于同一语义空间，相似度计算才有意义。

同时要存储**元数据**：chunk 来自哪份文档、原文在哪一页、文档创建时间等。元数据在过滤检索结果时非常重要，比如"只看最近三个月的文档"。→ **详见第 4 章**

⑤ 存入向量数据库

向量和元数据分别存入向量数据库（Milvus、Weaviate、Chroma、Pinecone 等）和普通数据库/文档存储。向量库的核心能力是**近似最近邻搜索（ANN）**：数百万向量中毫秒级找到与查询向量最相似的 top-K。→ **详见第 5 章**

2.3 在线阶段逐段拆解

```
用户 Query
   ↓
Query 改写 / 扩展（可选，虚线框 = 可选优化）
   ↓
向量检索（余弦相似度，召回 top-K）
   + 关键词检索（可选，BM25，混合召回）
   ↓
召回结果池（top-K 候选 chunks）
   ↓
精排 Rerank（可选，Cross-encoder 重新评分）
   ↓
最终 Context → 交给 LLM
```

① Query 处理（可选优化）

原始问题不一定适合直接检索：

- **Query 改写**：把口语化问题转成更适合检索的形式，或把复杂问题拆成几个子问题分别检索。例："我们公司的报销流程和上限分别是什么" → 拆成两个独立检索任务。
- **Query 扩展**：做同义词扩展，提高召回覆盖面，避免因用词差异漏掉相关文档。

> 这些是可选的优化，基础实现可以先跳过。

② 检索（Retrieval）

Query 向量化后，与向量库里所有 chunk 向量做相似度计算（通常用**余弦相似度**），召回 top-K，K 通常取 **3~10**。

更完整的实现会做**混合检索（Hybrid Search）**：同时跑向量检索（语义相似）和关键词检索（精确匹配，如 BM25），再合并两路结果，兼顾两种优势。

③ Rerank（精排）

初步召回的 top-K 相关性不一定都高。Rerank 在召回之后加一道精排：用专门的 **Cross-Encoder** 模型对 (Query, Chunk) 对打分，按新分数重新排序，只保留最相关的几条。

> Rerank 是 RAG 优化里**最常见也最有效**的手段之一，代价是多一次模型推理的延迟。→ **详见第 4.4 节、第 7 章**

④ 上下文构建（Context）

把最终筛选出的 chunk，加上元数据（来源文档、页码等），按一定格式拼装成上下文，连同用户原始问题构建最终 Prompt。典型结构：

```
你是一个企业知识库助手。请根据以下资料回答问题，如果资料中没有相关信息，请明确说明。

参考资料：
[来源：产品手册第3页]
…（chunk 内容）…

用户问题：我们的退款政策是什么？
```

⑤ 生成（Generation）

LLM 接收完整 Prompt，基于提供的上下文生成回答。

> **关键点**：Prompt 里要有明确的引导指令——让模型**优先依据资料回答**，而不是依赖自身参数知识，并要求在答案里标注来源。→ **详见第 6.3 节"缺少引用约束"**

2.4 三条核心认知（必须记住）

认知一：每个环节都影响最终质量，但影响方式不同。

环节	决定什么
Chunking	"能不能检索到相关内容"
Embedding 模型	"语义理解是否准确"
Rerank	"召回的 top 结果是否真的最相关"
Prompt 设计	"模型是否能正确利用上下文"

> 优化 RAG 本质上是**找到当前系统的薄弱环节**，而不是无差别地调参。

认知二：离线和在线必须保持一致。

Embedding 模型、文本清洗方式，离线建索引时怎么做，在线检索时就必须一样。索引是用 A 模型建的、检索时用 B 模型，两边向量空间不同，相似度计算完全失效。

认知三：检索的目标是精准，不是全面。

召回 top-3 的高质量 chunk，通常比召回 top-20 的混杂结果更好。**上下文越长，模型注意力越分散，信噪比越低。**

2.5 本章常见误区

误区	为什么错
"RAG = 向量检索"	向量检索只是在线侧的一个步骤。完整 RAG 还包括文档解析、Chunking 策略、Embedding 选型、元数据管理、Rerank、Prompt 设计等工程工作，**缺少任何一环都会拖累整体效果**
"只要模型够强，Chunking 随便切就行"	Chunking 是 RAG 最底层的基础设施。模型再强，检索到的 chunk 要么太短（语义不完整）要么太长（信息被稀释），生成质量都会大打折扣。**模型能力无法弥补检索质量的缺陷**
"Rerank 一定要加"	Rerank 有代价：多一次模型调用意味着更高延迟和成本。对实时性要求高、或文档量较小的场景，精准的 Embedding + 合理的 top-K 往往已足够。**先评估是否真需要，再决定是否加**

2.6 面试怎么问

Q：请描述一个完整的 RAG 链路。

参考思路：分两段回答。

- *离线侧*：文档加载 → 清洗预处理 → Chunking → Embedding 向量化 → 存入向量数据库（同时存元数据）。

- *在线侧*：用户 Query → （可选 Query 改写）→ Query 向量化 → 向量检索召回 top-K →（可选混合检索、Rerank 精排）→ 拼装 Context + Prompt → LLM 生成 → 输出带来源引用的答案。

Q：RAG 里哪些环节最影响效果?

参考思路：离线侧最关键的是 **Chunking 策略**（决定检索粒度）和 **Embedding 模型选型**（决定语义理解质量）；在线侧最关键的是 **Prompt 设计**（引导模型正确利用上下文）。其中**文档质量是前提**，再好的系统也处理不好乱码和结构混乱的输入。

Q：Embedding 模型在离线和在线需要保持一致吗？为什么?

参考思路：**必须一致**。Embedding 把文本映射到高维向量空间，不同模型的向量空间不同。离线用 A 建的索引、在线用 B 生成 Query 向量，两者处于不同空间，余弦相似度计算完全失去意义，检索结果会非常混乱。

2.7 本章关键结论

> RAG 不是一个算法，而是**一条工程流水线**。把这条链路想清楚了，后续讨论任何一个具体优化点——Chunking 怎么切、Embedding 怎么选、Rerank 怎么用——**都有了明确的落脚位置**。

第 3 章 切片策略 Chunking：决定检索质量的地基

3.1 用一个比喻建立直觉

你在准备一场考试，有一本 500 页教材。你的策略是：把书撕成小卡片，每张写一个知识点，考前根据题目抽卡片。

- 卡片切得太细（每张一句话）→ **上下文不完整**；
- 卡片切得太粗（每张一整章）→ 信息完整，但要看的太多，**真正需要的那句话藏在一堆无关文字里。**

RAG 的 Chunking 面对的就是这个核心矛盾：

> **chunk 太小，单个片段语义不完整；chunk 太大，检索出来的内容里信噪比太低。**

3.2 四种主流切分策略对比

策略	做法	优点	缺点	适用
① 固定长度切分 Fixed-size	按字符数或 token 数固定切，如每 512 token 一刀	实现极简，不需要理解文档结构	**完全不管语义边界**，可能把一句话从中间切断，把完整论述拆成两半	仅用于**原型阶段快速验证**，不适合生产
② 递归字符切分 Recursive Character	按优先级尝试不同分隔符：先段落（`\n\n`）→ 再换行（`\n`）→ 再句号，递归到每个 chunk 都在目标大小内（LangChain `RecursiveCharacterTextSplitter`）	倾向在**自然语义边界**断开，同时能控制上限大小	不理解深层语义	**普通文本场景的最佳默认选择**
③ 语义切分 Semantic	按句子分割，用 Embedding 计算相邻句子语义相似度，**在相似度明显跌落处切开**（话题转换点）	**语义完整性最好**，每个 chunk 主题相对聚焦	需对全文做 Embedding 计算，**慢且贵**；切出的 chunk 大小很不规则	文档质量要求极高、话题切换频繁的场景
④ 结构感知切分 Structure-aware	按文档固有层级切：Markdown 按 `#` / `##` / `###` 标题；法律文件按条款编号；代码按函数/类边界；PDF 按页码与章节标题	每个 chunk **天然对应一个有意义的主题**，检索精准度往往最高	依赖文档本身有清晰结构	**文档有明确层级时的最优选择**

同一段文本，四种策略切出来的差异：

- 固定长度：chunk 1 / 2 / 3 / 4 —— 机械等分
- 递归字符：chunk 1 / 2 / 3 —— 段落对齐
- 语义切分：chunk 1（话题 A）/ 2（话题 B）/ 3（话题 C）—— 话题边界
- 结构感知：第一节标题 + 该节正文 / 第二节标题 + 该节正文 —— 标题对齐

3.3 Overlap：被低估的关键参数

设想一段文字，某句话恰好落在两个 chunk 的边界：前半句在 chunk 1，后半句在 chunk 2。用户提问相关内容时，单独检索到 chunk 1 或 2，拿到的**都只有一半**，信息不完整，生成的答案就会出错。

Overlap 的解决思路：相邻 chunk 之间保留一段重叠内容。比如 chunk = 512 token、overlap = 64 token，那么 chunk 2 的前 64 token 与 chunk 1 的后 64 token 完全相同。这样边界附近的信息在两个

chunk 里都有备份，不管命中哪个都能拿到完整上下文。

示例：

```
无 overlap —— 边界信息被切割
chunk A: …该材料熔点较低，在常温
chunk B: 下即可熔化，因此…

有 overlap —— 边界信息在两侧都有保留
chunk 1: …该材料熔点较低，在常温
chunk 2: 在常温下即可熔化，因此…
         ↑ 重叠区域（两个 chunk 都包含）
```

代价：索引体积增大（overlap 内容被存两次）；检索结果可能出现重复。

> **推荐值：overlap 为 chunk 大小的 10%~20%。**

3.4 Chunk 过大、过小各有什么问题?

Chunk 过小：

- 单个 chunk **语义不完整**，孤立一句话往往没有足够上下文让模型理解。例如检索结果出现："是的，这种做法符合规范。"——符合什么规范？完全不知道。
- 切出大量 chunk，**索引规模膨胀，检索速度变慢。**

Chunk 过大：

- 每个 chunk 包含话题太多，**与用户问题的相关度被稀释。**假设 chunk 是 2000 token，用户只关心其中 50 token，这 2000 token 都会被塞进 Context，占用宝贵窗口，模型注意力难以聚焦。
- chunk 越大，**向量表示越"平均化"，语义越模糊**，检索精准度越低。

3.5 Chunk 大小经验值

场景	参考范围
对话型问答	256 ~ 512 token
知识库检索	512 ~ 1024 token
长文档摘要	1500 ~ 2000 token

> 这些都是估计值，必须根据**实际业务效果**调整。没有通用的最优大小。

3.6 实际项目里怎么选策略（决策框架）

情况	推荐策略
文档有清晰结构（章节、标题、条款）	**结构感知切分**，按自然结构边界切，语义完整性最好
普通散文、没有明确结构	**递归字符切分**，配合合适的 chunk size 与 overlap
文档质量要求极高、话题切换频繁	考虑**语义切分**，代价是更多计算资源和时间
快速验证原型	**固定长度切分**够用，**记得上线前换掉**

3.7 面试怎么问

Q：你们项目里 chunk 是怎么切的？

参考思路：**先说选择了什么策略及原因，再说具体参数（chunk size、overlap），最后说是如何评估和调整的。**

示范话术："我们的文档是有章节结构的产品手册，所以用了结构感知切分，按二级标题划分。每个 chunk 控制在 600 token 以内，overlap 设了 80 token。后来发现部分技术规格内容跨段落，把 overlap 调大到 120 token 后，相关问题的召回率有明显提升。"

Q：chunk 过大过小分别有什么问题？

参考思路：过小 → 语义不完整，单个 chunk 缺乏上下文，向量表示语义模糊，检索精准度低。过大 → Context 噪声增多，模型注意力被分散，且每个 chunk 的向量是多话题混合的平均，语义匹配变差。**两个方向都直接影响最终生成质量。**

Q：为什么要设置 overlap？

参考思路：overlap 是为了**防止重要信息落在 chunk 边界被割裂**，通过让相邻 chunk 共享一段内容，保证边界附近的信息在两侧都能被完整检索到。

3.8 本章关键结论

> Chunking 是 RAG 系统里最"不起眼"却影响深远的设计决策。它没有那么明显的技术感，但**糟糕的 Chunking 会让后面所有优化努力都打折扣**——检索到的片段不合适，再好的模型也生成不出好答案。

第 4 章 Embedding：语义压缩、模型选型，与 Rerank 的区别

4.1 一个经典对话引入问题

> 面试官：用户问"出差能不能报销坐飞机"，你的知识库明明有答案，为什么搜不出来？
> 候选人：可能是关键词没匹配上，我再多加几个同义词。
> 面试官：文档里写的是"民用航空器"，不是"飞机"，再想想。
> 候选人：是不是搜索词库不够全？
> 面试官：你加再多同义词也没用。你的检索到底在匹配什么？是**文字**，还是**语义**？

传统搜索在拼文字，而 Embedding，就是让机器读懂人类的语言。

4.2 Embedding 到底是什么

Embedding 是一种将文本（或图片、音频）映射到**高维向量空间**的技术。一个句子经过 Embedding 模型处理后，变成一个由几百到几千个浮点数组成的数组，例如 `[0.23, -0.87, 0.14, ..., 0.56]`（维度通常是 384、768 或 1536 维）。

关键不在于这串数字本身，而在于它在空间里的"位置"：

- 语义相似的文本，向量在高维空间里**彼此距离近**；
- 语义不相关的文本，**距离远。**

> 这是 Embedding 的核心承诺：**用空间距离来表达语义相似度。**

把真实的高维向量投影到二维平面，语义聚类的模式依然清晰：

```
        出行报销类                    动物类
[可乘坐民用航空器]              [猫的品种介绍]
[差旅费可报销]                  [哺乳动物的特征]
[出差交通补贴规定]              [狗的行为习性]
      ↑ 距离近，相关度高
★ 用户 Query：出差坐飞机能报销吗？

                        编程类
                [Python 语法示例]
                [函数调用方法]
                [代码调试的技巧]
```

用户 Query 向量化后，系统计算它与所有文档向量的距离，找到最近的几个——**这就是向量检索的本质。**

4.3 Embedding 不是生成文本，是在压缩语义

这是一个经常被混淆的概念：

	生成式大模型（GPT/Claude）	Embedding 模型
目标	**输出文字**，给一段输入生成下一段内容	**输出向量表示**，把语义压缩成固定维度的数字数组
保留	措辞、格式、风格	**语义信息**，丢掉格式和措辞细节
效果	生成连贯文本	**同一个语义，换各种说法，向量应该非常接近**

> 这就是语义搜索能跨越措辞差异找到相关内容的根本原因。

4.4 Embedding 模型怎么选（四个维度）

维度	说明	工程含义
向量维度	维度越高，理论上能表达的语义越细腻，但存储开销和检索速度成本上升。常见：384（轻量）、768（均衡）、1536（高精度）	大多数中等规模业务选 **768 维**是合理起点
最大 token 限制	每个模型都有输入长度上限，超限内容会被截断	**直接约束 chunk 的最大大小**。如 `bge-small-zh` 最多 512 token，chunk 就不能超过。选模型前必须核对
中文支持	中文 RAG 场景**最关键**的选型依据。通用英文 Embedding（如 `text-embedding-ada-002`）对中文支持明显不如专门优化中文的模型	中文场景口碑较好的开源选项：**BGE 系列、M3E 系列**
是否支持非对称检索	RAG 里典型非对称场景：Query 是短问题（"退款政策是什么"），文档片段是长说明文字	有些模型为这类 Query-Document 对专门优化。如 BGE 系列支持给 Query 加前缀 `Represent this sentence for searching relevant passages:` 来提升检索效果

4.5 Embedding 和 Rerank 有什么区别（高频面试题）

一句话记忆：Embedding 负责"粗筛"，Rerank 负责"精排"。

	Embedding	Rerank
模型结构	双塔模型（Bi-Encoder）	交叉编码器（Cross-Encoder）
编码方式	Query 和每个文档片段**各自独立**编码成向量	把 Query 和候选片段**拼接在一起**作为整体输入
打分方式	通过向量距离（余弦相似度）衡量相关性	模型直接输出一个相关性分数
能否预计算	**能**——文档向量可离线预计算存入向量库，检索时只算 Query 向量	**不能**——每次都需要逐对推理
速度与规模	毫秒级，可处理**百万量级**文档	慢，只适合对**少量候选（top-20 以内）**做精排
精度	较低（独立编码，看不到 Query 与文档的完整交互）	**更高**（能看到两者的深度交互）
在链路中的位置	粗筛：从海量文档中召回候选	精排：对候选结果重新评分

流水线关系：

> 全量文档 ──Embedding 召回 top-K（20~50）──> 候选池 ──Rerank 精选 top-N（3~5）──> 传入 LLM Context
> 粗筛 精排

4.6 中文场景和通用场景的区别

差异点	说明
分词边界	中文没有天然空格分词，字符级语义表达与英文差异显著；通用模型对中文的语义捕捉明显弱于专门训练的中文模型
专业术语	中文业务场景大量出现行业专业词（法律条款、医疗术语、金融名词），通用模型往往无法精准表达其语义关系
评估基准	选中文模型时，**MTEB（Massive Text Embedding Benchmark）中文 Retrieval 子任务榜单**是重要参考

选型建议：

- 中文业务为主 → 优先 **BGE 系列**或 **M3E**，而不是 `text-embedding-ada-002`（后者中文支持相对偏弱）；
- 需要同时处理中英文双语文档 → `bge-m3` 是较好选择（专门针对多语言检索优化）。

4.7 四大误区

误区	纠正
"Embedding 模型越大越好"	不一定。大模型精度更高但推理更慢、内存占用更大。实时检索场景下，一个轻量但专门为检索优化的模型（如 `bge-small-zh`）可能效果更好、成本更低
"用 OpenAI 的 Embedding 就够了"	`text-embedding-3-small/large` 英文极好，但中文场景建议做评测对比。很多中文业务用专门的中文模型效果更好，还能减少第三方 API 依赖、降低成本
"Embedding 和 Rerank 用不同模型没关系"	Embedding 模型选择没有强制约束，但 Rerank 模型最好与 Embedding 在训练数据分布上相近，否则评分口径不一致，**精排效果反而不稳定**
"换了 Embedding 模型不需要重建索引"	**完全错误。**不同模型的向量处于不同语义空间，旧索引向量与新模型 Query 向量不可比。**换模型必须重新对所有文档做 Embedding，重建整个向量索引**

4.8 面试怎么问

Q：Embedding 模型怎么选?

参考思路：从四个维度回答。① **语言场景**——中文业务优先选中文优化模型（BGE、M3E），不要直接用英文通用模型；② **输入长度**——模型最大 token 限制直接决定 chunk 大小上限；③ **向量维度**——根据存储资源和精度要求权衡，768 维通常是合理起点；④ **任务类型**——非对称检索（短 Query 找长文档）和对称检索（相似问题匹配）的最优模型不同，要看具体业务。

Q：Embedding 和 Rerank 有什么区别?

参考思路：Embedding（双塔/Bi-Encoder）对 Query 和文档**分别独立**编码成向量，通过向量相似度召回，优点是文档向量可离线预计算、检索极快、可处理百万量级，适合**粗筛**；Rerank（Cross-Encoder）把 Query 和每个候选文档**拼接后联合**输入，直接输出相关性分数，模型能看深度交互、精准度更高，但无法预计算、需逐对推理，只适合对**少量候选精排**。两者是流水线关系：Embedding 粗筛召回候选池，Rerank 精排过滤出最终传给 LLM 的上下文。

4.9 本章关键结论

> Embedding 是 RAG 里"让机器理解语义"的核心基础设施。它不是把文字变成数字的简单编码，而是**把语义压缩进一个可计算的空间**，让相似的意思在空间里相互靠近。

第 5 章 向量数据库：ANN 索引原理与选型

5.1 从一个具体问题出发

你做了一个 RAG 系统，有 100 万个文档 chunk，每个 chunk 经 Embedding 变成 768 维向量。用户提问后，你要做的是：**找到 100 万个向量里和 Query 向量最相似的 top-5。**

最直接的方法是**暴力搜索**：逐一计算余弦相似度、排序、取 top-5。在一台普通服务器上大概需要**几百毫秒到几秒**——对实时对话完全不可接受。向量数增长到 1000 万、1 亿时，耗时线性增长，根本无法用于生产。

> 这就是**向量数据库要解决的核心问题**：在海量高维向量中，如何在毫秒级内找到最相似的结果？

5.2 为什么不能用 MySQL 做向量检索（高频面试题）

答案不只是"MySQL 不支持"，而是有本质的技术原因。**两个层面：**

层面一：功能层面的模型差异

	传统数据库	向量数据库
数据	结构化，可枚举	高维浮点向量
查询	离散条件匹配（`city='北京' AND age>25`）	几何空间距离
索引	B-Tree / Hash Index	HNSW / IVF 高维空间近似最近邻
结果	精确匹配的行（要么有，要么没有）	相似度排名 top-K
核心挑战	快速定位符合条件的行	海量高维向量快速检索

传统关系型数据库的查询本质是**精确匹配或范围扫描**，通过 B-Tree 索引快速定位符合条件的行。而向量相似度搜索问的不是"哪些向量**等于**某个值"，而是"哪些向量和查询向量**最接近**"——这是**高维连续空间里的几何问题**。B-Tree、Hash Index 这类索引结构设计之初就没有考虑"高维几何相似度"这个维度，完全无效。

层面二：性能层面

即便 PostgreSQL 的 `pgvector` 扩展支持向量存储，大规模场景下性能仍远不如专门的向量数据库。全量计算余弦相似度的复杂度是 **O(N)**，百万级向量时远远无法满足毫秒级需求。向量数据库通过 HNSW、IVF 等**近似索引结构**，把检索复杂度降低到**近似 O(log N)**。

5.3 ANN：如何在速度和精度之间取得平衡

关键概念是**向量索引**。向量数据库不做暴力全量计算，而是**预先构建特殊索引结构**，让检索时只需访问一小部分向量就能找到近似的最近邻——这就是 **ANN（Approximate Nearest Neighbor，近似最近邻搜索）**。

> **"近似"两字的含义**：ANN 不保证找到绝对最近邻，但实践中结果精准度足够高（通常 **95%+**），而速度提升是**数量级**的。

① HNSW（Hierarchical Navigable Small World）

构建层级结构的图：

- **最顶层**：少量节点，形成"高速公路"；
- **底层**：所有节点，形成"街道网络"。

检索时从顶层入口进入，沿邻居边快速导航，逐层下降，**像 GPS 导航一样越来越精细**，最终定位到目标附近的最近邻。

优点	缺点
查询速度极快、精度高；**Milvus、Qdrant、Weaviate 等主流向量库的默认索引**	构建索引时**内存占用较大**

② IVF（Inverted File Index）

先对所有向量做 **K-Means 聚类**，把相似向量分组到同一个"货架"里。检索时先找到 Query 向量最近的几个货架，只在这些货架内做精细查找。

优点	缺点
内存占用较低，适合超大规模数据集	若 Query 向量恰好落在两个货架边界，**可能漏掉相关结果（边界效应）**

HNSW vs IVF 对比：

	HNSW	IVF
结构	分层可导航图	聚类倒排（K-Means 分桶）
查询速度	极快	快
精度	高	较高，有边界效应
内存	高	低
适用	通用、精度优先	超大规模、内存受限

5.4 向量数据库还能做什么（三个工程能力）

① 元数据过滤（Metadata Filtering）

每个向量可附带结构化元数据：文档来源、创建时间、部门归属、文档类型等。检索时可在向量相似度搜索的同时叠加元数据条件——"只在最近三个月的文档里搜索""只搜索产品部门的文档"。这让 RAG 能做**语义相关 + 业务过滤**的组合查询。

② 混合检索（Hybrid Search）

主流向量库（Weaviate、Elasticsearch with dense vectors）支持同时运行向量搜索和关键词搜索，再用融合算法（如 **RRF，Reciprocal Rank Fusion**）合并两路结果。这对**中文专业术语场景尤其有用**，因为有些专有名词的语义表示很弱，必须依赖关键词精确匹配。

③ 向量更新和删除

知识库需要持续更新——文档修改、删除了，向量索引也要对应更新。不同向量库对这个操作的支持成本差异很大，**选型时需要考虑业务的更新频率。**

5.5 主流向量数据库与选型建议

三类选择：

类别	代表	特点
专用向量数据库	Milvus、Qdrant、Weaviate、Pinecone	从设计之初以向量检索为核心，功能最完善、性能最优，适合大规模生产
传统数据库的向量扩展	PostgreSQL + pgvector、Elasticsearch + dense vectors	已有这些基础设施时，小规模可用；百万量级时性能明显不如专用方案
轻量级本地库	FAISS、Chroma	FAISS 是纯向量索引库（非完整数据库），适合嵌入应用做原型或小规模部署；Chroma 是轻量级向量数据库，适合开发调试

选型建议表：

规模 / 需求	推荐
几十万向量以内	Chroma 或 pgvector 足够
百万以上，需要元数据过滤和混合检索	Milvus 或 Qdrant
云端托管、免运维	Pinecone

5.6 四大误区

误区	纠正
"向量数据库结果不精确，所以不可靠"	ANN 的"近似"指不一定是数学意义上的绝对最近邻，但实践中召回精度通常 **95%~99%**（用 `recall@K` 衡量）。对 RAG 完全够用，而速度收益是数量级的
"向量库越大越好，建索引参数越高越好"	HNSW 有超参数（如 `M` ——每节点最大连接数、`efConstruction` ——建图搜索深度），越大精度越高，但内存占用和构建时间成比例增加。**需要找精度/速度/内存的平衡点**
"向量相似度高就代表语义相关"	向量相似度是 Embedding 模型**学出来的**相似度，取决于模型训练质量和领域覆盖。若模型对某专业领域支持不足，即便相似度高，语义也可能并不相关。**向量数据库只是工具，底层模型质量仍是决定因素**
"用 FAISS 就是用了向量数据库"	FAISS 是**向量索引库**，不是完整数据库系统：没有持久化、没有元数据管理、没有 CRUD 接口，只负责索引构建和搜索计算。生产环境需自己封装持久化和业务逻辑，或直接用完整的向量数据库

5.7 面试怎么问

Q：为什么不用 MySQL 做相似检索?

参考思路：分两个层面。① **功能层面**：传统关系库的查询模型是精确匹配和范围扫描，基于 B-Tree 等索引结构，无法支持"找高维空间中的近似最近邻"这种几何问题；② **性能层面**：即便通过扩展支持向量存储和计算（如 pgvector），百万级向量时全量计算余弦相似度复杂度是 O(N)，远无法满足毫秒级检索。向量数据库通过 HNSW、IVF 等专门近似索引，把复杂度降低到近似 O(log N)。

Q：向量数据库为什么能加速召回?

参考思路：核心是 **ANN（近似最近邻搜索）**，通过预先构建向量索引避免全量计算。以 HNSW 为例，它构建分层可导航小世界图，检索时从顶层稀疏图入口快速导航，逐层定位到目标区域，只访问极少数节点就能找到近似最近邻，速度比暴力搜索快几个数量级。代价是"近似"——不保证绝对最近邻，但实践中 recall 精度 95%+ 足以满足 RAG 需求。

5.8 本章关键结论

> 向量数据库解决的是一个传统数据库**根本没有设计来解决的问题**：在海量高维向量中快速找到相似的那些。

第 6 章 排障诊断：检索侧五类 + 生成侧四类问题

6.1 总原则：先分流，再定位

> 一位民间大神的话："你连问题出在哪都没定位，优化全靠瞎试。"

很多人一碰到 RAG 答不准，第一反应就是换模型、上 Rerank，却从不去拆解根源。

RAG 只有两段关键链路：**离线构建索引、在线检索生成。**答案不好，要么**检索崩了**（没找到、找错了），要么**生成崩了**（找到了但没用好）。

> **不先定位是检索还是生成的问题，所有优化都是治标不治本。**

问题分类总览：

```
RAG 完整链路
├─ 检索侧问题
│    ├─ 文档质量差（噪声、乱码、格式混乱）
│    ├─ chunk 粒度不合理（太粗或太细、语义割裂）
│    ├─ Embedding 选型不当（中文弱、领域不匹配）
│    ├─ 召回不足（相关文档没被检索到）
│    └─ 缺少 Rerank（召回结果精排不足）
└─ 生成侧问题
     ├─ Prompt 拼接有问题（格式混乱、指令不清）
     ├─ 上下文干扰（无关 chunk 稀释注意力）
     ├─ 缺少引用约束（模型超出资料自由发挥）
     └─ 模型理解能力弱（无法从资料推断答案）
```

6.2 检索侧五类问题

#	问题	机理	典型症状	应对思路
1	**文档质量差**	输入是扫描件 OCR 乱码、格式混乱的 PDF 解析结果、或充满模板噪声的 HTML；**向量化质量决定检索质量**，再好的 Embedding 也无法从噪声中提取有效语义	检索结果里出现大量无意义片段："第 3 页""版权所有""……"	Chunking **之前**加强文档预处理：清除页眉页脚、过滤模板噪声、对低质量文档做专项清洗甚至人工校对
2	**Chunk 粒度不合理**	太小 → 每个片段只有一两句，语义不完整，向量无法准确表达主题；太大 → 片段含多个话题，向量变成多主题混合，与 Query 语义匹配变模糊	命中相关话题但内容不完整；或召回精准度持续下降	按文档结构选策略；用评估集对比不同参数的 Recall@K（见第 3 章）
3	**Embedding 选型不当**	模型语义理解能力决定"相似度计算的语义准确性"。用了对中文支持弱、或对某专业领域覆盖不足的模型，语义相似的内容在向量空间里距离很远	相同的中文提问，换了说法检索结果差异巨大；同义句子得分很低	评估几个候选模型（参考 **MTEB 中文榜**），在**业务数据**上跑召回率对比实验，选最适合当前领域的
4	**召回不足**	索引数据问题导致相关文档根本未被检索到：文档更新了但索引没及时重建、部分文档入库时被错误过滤、top-K 设置太小导致相关内容排在 K 之外	知识库明明有答案但系统给不出；**手动搜索能找到，自动检索找不到**	检查索引覆盖是否完整；适当增大 top-K（如 5 → 10~20）再配合 Rerank 过滤；引入混合检索（向量 + 关键词）作为补充
5	**缺少 Rerank**	初步召回的 top-K 未必按真实相关性排序——**向量相似度和"内容与问题的真实相关性"是两个不同维度。**有些片段向量很近（讨论了相关话题），但未必精准回答了具体问题	最终答案"相关但不精准"，方向对但细节错；Context 里明明有答案，模型拼接出的回答却不够准确	召回后加 Cross-Encoder Rerank，对 (Query, Chunk) 对精细评分，只保留真正高相关的结果传入 LLM

6.3 生成侧四类问题

#	问题	机理	典型症状	应对思路
1	**Prompt 拼接有问题**	拼接方式不当：多个 chunk 之间没有分隔符、来源信息缺失、指令和内容混在一起、或指令表述模糊	模型回答时把多个文档内容混淆；对"资料中没有答案就说不知道"这类指令遵从率很低	明确区分"系统指令"和"参考资料"，为不同来源内容加分隔符，在指令里明确如何处理资料不足的情况
2	**上下文干扰（Context Noise）**	召回的 chunk 不是每条都高度相关，低相关 chunk 混入 Context，**稀释模型注意力**，让模型在大量无关信息中"迷失"。在没有 Rerank 的系统里尤为突出，也与 top-K 过大有关	答案混入奇怪的、不相关的细节；模型给出综合了多个不相关来源的"平均答案"	用 Rerank 过滤低相关 chunk；**减少传入 Context 的 chunk 数量（宁少勿滥）**；Prompt 里明确要求只根据提供的资料作答
3	**缺少引用约束（模型自由发挥）**	Prompt 没有明确限制"只基于提供的资料作答"，模型会自然地把**自身参数知识**和检索内容混合起来回答	答案看起来比实际上更完整，但准确性存疑，引入幻觉	系统 Prompt 里加明确引用约束："请严格根据以下资料回答，资料中没有的内容请明确说明无法回答"，并要求标注来源
4	**模型理解能力不足**	即便检索质量很高，模型自身推理能力也会成为瓶颈——尤其当答案需**跨多个 chunk 综合推断**、或问题本身需要复杂逻辑推理时。小模型尤为明显	检索结果正确但答错；跨 chunk 综合类问题几乎全军覆没	推理密集型问题升级到更强的模型；优化 Prompt，明确要求逐步推导；或通过 **Query 拆解**把复杂问题分解成多个简单检索任务

6.4 如何系统地诊断 RAG 问题（四步法）

第一步：判断是检索问题还是生成问题。

最简单的方法：**手动查看检索结果。**把用户的问题直接查向量库，看返回的 top-K chunk 里有没有包含正确答案。

- top-K chunk 里**有**正确答案，但最终生成的回答是错的 → **生成侧问题**
- top-K chunk 里**根本没有**正确答案 → **检索侧问题**

第二步：在检索侧进一步定位。

- **召回率评估**：对一批"已知有答案"的问题，统计正确 chunk 是否出现在 top-K 里（**Recall@K**）。如果 Recall@K 低，进一步分析是 **Embedding 问题**（语义匹配差）还是**文档覆盖问题**（相关内容根本没有入库）。
- **检索结果质量评估**：查看 top-K 的相似度分数分布——分数普遍很低 → Embedding 语义空间不匹配；分数高但内容无关 → Embedding 模型在当前领域理解有偏差。

第三步：在生成侧进一步定位。

把正确的 chunk **手动构造 Prompt** 喂给模型，看模型能否正确回答。能 → 说明是检索问题（找对了内容模型能用好）；不能 → 说明是 Prompt 设计或模型能力问题。

第四步：优化方向分流。

分流结果	优化方向
检索侧问题（内容没被召回）	换 Embedding / 修 Chunk / 加混合检索 / 补文档
生成侧问题（有内容但没用好）	改 Prompt / 加引用约束 / 加 Rerank / 升级模型

诊断流程图（文字版）：

```
答案不准
  ↓
手动查看 top-K 检索结果：正确答案在 top-K 里吗？
  ├─ 没有 → 检索侧问题（内容没被召回）
  │          ↓ 计算 Recall@K 统计召回率
  │          ↓ 优化方向：换 Embedding / 修 Chunk / 加混合检索 / 补文档
  └─ 有    → 生成侧问题（有内容但没用好）
             ↓ 手动构造 Prompt 测试，验证模型能否利用资料
             ↓ 优化方向：改 Prompt / 加引用约束 / 加 Rerank / 升级模型
```

> **核心原则：先隔离检索侧和生成侧，再定位具体环节，避免盲目优化。**

6.5 面试怎么问

Q：你们 RAG 系统答不准，你会怎么排查？

参考思路：先从检索侧、生成侧做分析。**手动查看 top-K 检索结果**——如果答案在里面，说明是生成侧问题（Prompt、上下文噪声、引用约束、模型能力）；如果答案不在里面，说明是检索侧问题（召回率低、Embedding 选型、Chunk 质量、文档覆盖）。然后用 **Recall@K**（在 top-K 个检索结果中，正确 chunk 出现的比例）等指标量化，再针对性调整。

Q：如何评估 RAG 系统的召回效果？

参考思路：构造评估集——选取一批"已知有答案的问题"及其对应的正确 chunk，然后计算 **Recall@K**。K 通常取 5 或 10。一般而言，**如果 Recall@5 低于 70%，说明检索侧有明显问题**，需要优先优化。

6.6 本章关键结论

> RAG 答不准是一个**背后可能有很多不同病因的症状**。先有清晰的分类框架（检索侧 vs 生成侧），再有明确的诊断路径（从 top-K 结果的人工检查切入）。

第 7 章 优化思路：Query / 检索 / 精排 / Context 四侧

7.1 优化要对症下药

> **核心认知：RAG 优化没有万能药。** "加个 Rerank 就搞定了""换个更好的模型就行了"——听起来合理，但很多时候只是在用锤子找钉子。

正确姿势：**先诊断问题出在哪个环节（第 6 章），再选择对应的优化手段。**

优化手段三类（四侧）全景：

```
用户 Query
   ↓
[Query 侧优化]  Query 改写（扩展同义词） / Multi-Query（拆成多角度）
   ↓
[检索侧优化]    混合检索（向量 + 关键词） / 元数据过滤（时间、部门筛选）
   ↓
[精排侧优化]    Rerank（Cross-Encoder）
   ↓
[Context 侧优化] 父子块检索（小检索大使用） / Context 压缩（去冗余保关键）
   ↓
LLM 生成
```

7.2 Query 侧优化

① Query 改写（Query Rewrite）

用户的原始问题往往措辞口语化、表达多样、语义模糊，不一定适合直接做向量检索。思路是：**在检索之前，先用 LLM 对用户问题做一轮处理**——扩展同义词、修正表述、补全隐含条件，让改写后的查询更贴近知识库里文档的用词习惯。

例：用户问"这个合同能退吗" → 改写为"合同解除条款、退款政策、合同撤销条件"，在多角度下做检索，覆盖率明显更好。

② Multi-Query（多角度查询）

当一个问题可以从多个角度理解时，生成多个不同版本的 Query，分别检索，然后把多路结果合并去重。

例：用户问"我们产品的 SLA（服务水平协议）是多少"，可展开为：

- "服务等级协议内容"
- "故障响应时间"
- "可用性承诺"

- "赔偿条款"

每路独立检索，再合并结果，**召回率会显著提升。**

7.3 检索侧优化

① 混合检索（Hybrid Search）

纯向量检索擅长语义理解，但对**专有名词、精确匹配**（人名、产品型号、法规编号）不如关键词搜索准确。混合检索把两者结合：同时跑向量检索和 **BM25** 关键词检索，再用融合算法（最常用 **RRF，Reciprocal Rank Fusion**）合并排序。

> **RRF 的思路很直觉**：一个文档在两路结果里排名都靠前，它的融合分数就高；只在一路排名靠前的，分数打折。

② 元数据过滤（Metadata Filtering）

在向量相似度检索的同时，叠加结构化元数据条件："只搜索 2024 年之后的文档""只在合同类文档里搜索""只搜索研发部门的内部规范"。

> 元数据过滤**不改变检索算法**，但可以大幅缩小搜索范围，既提高精准度，又降低计算开销。

7.4 精排优化：Rerank

这是 RAG 优化里最常被提及的手段，但它的位置非常具体：**在向量检索召回 top-K 候选之后**，用 Cross-Encoder 对每个 (Query, Chunk) 对精细打分，重新排序，只保留真正高相关度的 top-N 传给 LLM。

核心价值：向量检索的相似度是**独立编码**的，不能完整捕捉 Query 和文档之间的深度语义关联；Cross-Encoder 能看到两者的**完整交互**，打分更精准。

混合检索 + Rerank 组合机制（工程上最成熟的检索优化组合）：

```
用户 Query（原始 / 改写后）
   ├─ Dense Retrieval（Bi-encoder 向量化 → ANN 近似搜索 → 余弦/点积相似度）→ Top-100 候选
   └─ Sparse Retrieval（BM25 词频打分 → 精确词汇匹配）→ Top-100 候选
                    ↓
         RRF 融合（Reciprocal Rank Fusion）→ 合并 Top_N 候选
                    ↓
    Cross-encoder Rerank（Query + Doc 拼接后联合打分）
        Q ⊕ Doc1 → 0.92
        Q ⊕ Doc2 → 0.45
        Q ⊕ Doc_n → 0.78
                    ↓
        按相关性重排序 → Top-K Chunks（3~5 条）→ 注入 Prompt 生成
```

> 两路各召回一定量候选，经 RRF 融合形成候选池，再由 Cross-Encoder 精排，最终只有 **3~5 条**高质量 chunk 进入 Context。

Rerank 什么时候该加 / 不该加：

时机	判断依据
该加	Recall@K 已经足够好（相关内容确实被召回了），但答案精准度仍不高——召回了相关内容，但没有精确筛出最核心的部分
不急着加	Recall@K 本身很低，相关内容根本没被召回——这时加 Rerank 没有意义，**应先解决召回问题**

7.5 Context 侧优化

① 父子块检索（Parent-Child Retrieval）

解决 Chunking 里的经典矛盾：小 chunk 利于精准匹配（向量更聚焦），但传给模型时上下文不够完整；大 chunk 上下文完整，但检索精准度差。

思路：建立两级索引。

- 用**小 chunk（子块）做检索**，找到相关内容；
- 实际**传给 LLM 时使用包含该子块的大 chunk（父块）**。

例：一篇 2000 字文章按段落切成 5 个 400 字子块做索引，但匹配到某个子块后，传给模型的是整篇文章（或其中一个更完整的部分）。**兼顾检索精准度和上下文完整性。**

② Context 压缩（Context Compression）

当召回的 chunk 里包含大量与当前问题无关的内容时，用一个 LLM 先做压缩提取——让模型阅读 chunk，把与 Query 高度相关的句子提取出来，丢弃无关内容，再把压缩后的片段拼成最终 Context。

> 能显著减少上下文噪声，但会**额外增加一次 LLM 调用的延迟和成本**，适合对准确性要求极高、对延迟容忍度较高的场景。

7.6 初学者应该如何选择优化方向（四条原则）

面对这么多手段，最容易犯的错是"什么都想加"——加了 Query 改写、混合检索、Rerank、父子块，结果系统延迟飙升，性能变差，还搞不清到底哪个环节帮了忙。

理性的顺序：

1. **先建立 baseline，做好评估。** 在任何优化之前，先用 Recall@K 和答案质量指标（可用 LLM 打分或人工评估）建立**可量化基线**。没有基线，就无法判断优化是否有效。

2. **优先解决"能不能找到"，再解决"找到了对不对"。** 如果 Recall@5 很低，先优化召回（换 Embedding 模型、调整 Chunk 策略、加混合检索），而不是先加 Rerank。
3. **每次只改一个变量。** 同时改 Chunk 大小、Embedding 模型和是否加 Rerank，你永远不知道是哪个起了作用。
4. **按成本/收益决定优先级。** 混合检索（增加 BM25 索引）成本低、覆盖面广，通常是**性价比最高的第一步优化**；Query 改写、Rerank 会增加延迟和成本，要根据业务 SLA 决定是否值得。

7.7 面试怎么问

Q：你们 RAG 系统做了哪些优化?

参考思路：按优化框架（Query 侧 / 检索侧 / 精排侧 / Context 侧）分类回答，说清每个优化针对的具体问题、实施前后的效果变化、以及取舍（为什么做这个而不是那个）。

示范："我们发现 Recall@5 在 60% 左右，先引入了混合检索，Recall@5 提升到 78%；再加了 Rerank，答案精准率提升了约 15%。"——**有数据有逻辑才是好回答。**

Q：混合检索怎么实现？两路结果怎么融合?

参考思路：向量检索用 Embedding 模型和向量数据库；关键词检索用 BM25 算法（Elasticsearch 或向量库的关键词搜索功能）。融合用 **RRF（Reciprocal Rank Fusion）**：对每个文档，把它在两路排名中的名次取倒数相加，得到融合分数，重新排序。RRF 对超参数不敏感，实现简单，效果稳定，是工程首选。

7.8 本章关键结论

> RAG 优化是一个**按需组合、逐步迭代**的过程，而不是一次性把所有手段全加上。理解每种手段解决的是哪个具体问题，建立可量化的评估体系，优先解决最影响效果的瓶颈——这才是 RAG 优化的正确姿势。

第 8 章 长文档与代码检索：结构、版本、权限

8.1 三个硬问题

碰到几百页的需求文档、或几十万行的代码仓库，只知道"父子块"还不够。这里多了三个硬问题：

> **结构不能断、版本不能错、权限不能漏。**

典型面试对话：

> 面试官：你们的代码知识库怎么切片？怎么保证搜到的是当前分支，而且用户有权限看？
> 候选人：按 512 Token 切，加一点 overlap，再做向量检索。
> 面试官：这个回答适合原型，不适合真实系统。

为什么不够？**固定切片会把标题和正文、函数签名和函数体拆开；不记录分支与提交，可能拿旧代码回答；不在检索前做权限过滤，还可能把受限内容带进候选集。**

> 真正要解决的不是"Chunk 设多大"，而是：**怎么用小块找准，再按结构拿回完整、当前且有权限的证据。**

8.2 简要回答（先记住这五条）

- 长文档按**标题、章节、条款**建树；代码按**类、函数、方法等 AST 节点**切分；
- **子块负责检索精度，父块负责给模型补齐上下文**，两者用 `parent_id` 关联；
- 代码检索要**组合语义检索与关键词检索**，符号名、路径和错误码不能只靠向量；
- 每个块都要携带 `repo`、`branch`、`commit`、路径和 `ACL` 等元数据；
- 在线检索**先按身份和目标版本过滤，再召回、Rerank、父级扩展和组装 Context。**

> 一句话：**检索单位不等于返回单位；相关不等于可用；可用还必须满足完整、当前和有权限。**

8.3 固定切片为什么在长文档里容易失效

一份需求文档通常是"章节 → 小节 → 条款 → 表格"的层级。某个子块里只有"退款周期为 7 个工作日"，但适用范围可能写在父标题"企业版年付客户"里。

- 只返回子块 → **答案缺条件**；
- 返回整篇文档 → **噪声和 Token 太多。**

所以长文档**不能只保存文本，还要保存它在标题树中的位置**，例如：

> 产品手册 / 计费规则 / 企业版 / 退款政策

这条路径既能参与检索和 Rerank，也能在命中后帮助系统找到父章节、相邻条款和表格说明。

8.4 代码为什么不能按字符数硬切

代码的自然边界不是句号和换行，而是**模块、类、函数、方法和语句块。**

如果固定切片刚好从函数中间切开，Chunk 里可能只有实现、没有函数名参数返回类型；也可能只有调用点、没有被调用函数的定义。**向量能判断"语义相似"，却不知道这个片段属于哪个符号。**

更合适的做法：先用解析器生成语法树，再按 **AST 节点**切分（可以用 Tree-sitter 构建具体语法树，并在文件编辑后增量更新）。

工程中通常保留这些信息：

- 函数或方法**签名、注释和函数体**；
- 所属**类、模块、文件路径和符号全名**；
- **起止行号、语言、导入关系和被调用符号**；
- **仓库、分支、提交哈希与内容哈希。**

> **AST 负责"不把结构切坏"，但不负责解决所有检索问题。** README、配置、SQL、错误日志仍要用对应的结构解析器或文本切分策略。

8.5 Parent-Child 怎么兼顾找得准和读得懂

核心做法是建立**两级或多级节点**：

层级	内容	用途
Child	函数、短条款或小段落	用于 Embedding、关键词召回和 Rerank
Parent	类、章节或完整规则	最终返回给模型
Document	文件或整篇文档	只在跨章节任务中按需继续向上扩展

每个 Child 保存 `parent_id`，父节点保存在文档存储中。**命中 Child 后，不是机械返回整篇文档，而是根据问题取父节点、标题路径、必要的相邻节点。**

8.6 为什么版本和权限必须进入索引

代码不是一份静态文本。同一路径在 `main`、功能分支和历史 commit 上可能完全不同。Git 的分支会随提交移动，而 **commit 表示确定的快照。**

索引至少要保存：

```
{
  "repo": "payment-service",
  "path": "src/refund/RefundService.java",
  "symbol": "RefundService.apply",
  "branch": "feature/refund-v2",
  "commit": "a1b2c3d",
  "parent_id": "class:RefundService",
  "acl": ["team:payment"]
}
```

在线查询时，**用户问题必须和当前工作区范围一起进入检索**：目标仓库是什么、当前分支或 commit 是什么、调用者属于哪个租户和权限组。

> **原则：先过滤，后相似度召回。**
>
> 如果先召回再删权限不匹配的结果，受限内容仍可能进入候选缓存、Rerank 服务或调试日志；而且删除后候选数量不足，召回质量也会突然下降。

8.7 在线检索链路怎么跑（七步）

1. 从会话或 IDE 获取 `repo` + `branch/commit` + `user/tenant`；
2. 解析 Query，识别**符号名、文件名、错误码和自然语言意图**；
3. **先做版本、路径和 ACL 过滤**；
4. 同时跑**向量检索与 BM25**，合并候选；
5. 在 **Child 粒度 Rerank**，避免大父块稀释相关度；
6. 根据 `parent_id` 取父级、标题路径或必要的相邻节点；
7. 去重并按 **Token 预算**组装 Context，附带路径、行号和 commit 作为引用。

> **最容易写反的是第五步和第六步**：先在小块上排序，再向父级扩展。如果先把所有 Child 展开成大块再 Rerank，相关信号会被大量父级文本冲淡，延迟也会上升。

8.8 命中子块后，要不要总是返回整个父块？

不要。 父级扩展是一个**预算决策**，不是固定动作。

- 一个函数只有 30 行 → 可以返回整个函数；
- 一个类有 3000 行 → 只补签名、字段、目标方法和直接依赖；
- 一个章节只有两页 → 可返回完整章节；
- 一个章节几十页 → 补标题路径和相邻条款。

可以设置三个门槛：父块最大 Token、允许扩展的层级、相邻节点数量。超过预算时，**优先保留定义、约束、输入输出和直接依赖，不要粗暴截断末尾。**

8.9 项目里怎么落地和评估

先用最小方案建立 Baseline：结构化切分、Child 混合检索、Parent 补全、版本和 ACL 前置过滤。不要一开始就加调用图、摘要树和复杂 Agent。

评估集要专门覆盖五类问题：跨小节答案、精确符号查找、同文件不同分支、历史 commit、无权限访问。

除 Recall@K、MRR 和答案质量外，还要增加：

扩展指标	含义
父级补全正确率	命中 Child 后是否拿到了正确父节点
版本错误率	是否引用了目标 commit 之外的内容
越权召回率	无权内容进入候选的比例，**目标必须是 0**
Context 有效率	进入上下文的 Token 中有多少真正支持答案
索引新鲜度	代码更新到可检索之间的延迟

索引更新策略：用 `commit + path + content_hash` 判断新增、修改和删除，**只重建受影响节点及父子关系。**否则每次全量重建，代码库一大，成本和更新延迟都会失控。

8.10 知识拓展 Q&A

Q：有了 AST 切分，还需要关键词检索吗？需要。函数名、类名、错误码、配置键和路径是**精确符号**，BM25 或符号索引通常比纯向量更稳；自然语言描述和业务意图再交给向量检索。两路召回后统一 Rerank。

Q：Parent-Child 和增加 overlap 有什么区别？Overlap 只缓解相邻切片的**边界断裂**，不知道标题、类和函数的真实父子关系。Parent-Child 是**显式结构关系**，可以跨多个子块补回完整父级，但索引和存储更复杂。

Q：代码依赖是不是应该全部展开进 Context？不是。调用图可能迅速扩散。**先返回目标符号，再按问题补一跳直接依赖**；只有测试失败分析、跨模块重构等任务，才逐层扩展并设置深度与 Token 上限。

Q：多个分支会不会让索引膨胀？会。常见做法是**长期保留默认分支和活跃分支**，对历史版本按需构建或设置过期策略；相同内容可按内容哈希去重，但**版本、路径和 ACL 关系不能丢。**

8.11 本章关键结论

> 别再把"512 Token + overlap"当成长文档和代码检索的完整答案。**能证明返回的内容找得准、补得全、版本对、权限安全，才算把这套检索系统做完整。**

第 9 章 Agentic RAG：从固定流水线到动态检索循环

9.1 问题的起点

传统 RAG 处理"退款政策是什么"没问题。但问题一复杂，比如：

> **对比新旧两版退款政策，找出华东区的例外条款，再说明这些变化会影响哪些历史订单。**

一次检索就开始吃力了。它要查多个问题、多个版本、多个数据源，还要判断证据够不够。第一次没找到，不能硬着头皮往下答，得换个问法继续找。

> **传统 RAG 把检索写死在流程里，Agentic RAG 则把检索变成 Agent 可以按需使用、反复使用的工具。**

定义：Agentic RAG 不是某一个固定算法，也不是某个框架的专属功能，而是**一类架构模式**。在这类系统里，大模型不只负责最后写答案，还参与决定：**要不要检索、问题怎么拆、查哪个数据源、检索结果是否可用、要不要换个 Query 再查，以及什么时候可以停止。**

9.2 RAG 的四阶段演进（递进关系）

每往前走一步，都是因为上一步解决不了新的问题。

阶段	谁在决定检索路径	检索次数	出错后的行为
① **传统 RAG**（2020 年 RAG 论文思路的工程化）	开发者写死	通常 1 次	**带着错误结果继续生成**
② **Advanced RAG**（加 Query 改写、Multi-Query、混合检索、Rerank、父子块、Context 压缩）	开发者写死更多优化	固定若干次	依赖预设的改写、重排策略
③ **自适应 / 纠错 RAG**（Self-RAG、CRAG、Adaptive-RAG）	分类器或模型参与判断	0 到多次	评估、改写或补充检索
④ **Agentic RAG**	Agent 根据目标和状态决定	动态	换 Query、换工具、补证据或拒答

各阶段的隐含假设与局限：

- **传统 RAG** 的链路是：`Query → 检索 Top-K → 拼接 Context → LLM 生成`。它解决了大模型不知道私有数据、知识更新慢、答案无法溯源的问题。但它有一个隐含假设：**用户的一句话已经足够适合检索，而且第一次返回的 Top-K 就包含答案。** 现实里这个假设经常不成立。
- **Advanced RAG** 让检索召回更高、上下文更干净，但要看清边界：**流水线变强了，流水线本身仍然是固定的。** 开发者提前规定每个请求都改写三次、查两个索引、做一次 Rerank。即使一个简单问题根本不需

要检索，它也照跑；即使复杂问题一次没查全，它通常也不会自己增加下一轮。

- **自适应与纠错 RAG** 推动了同一个变化：**检索不再是必走一次的固定步骤，而是一个可以判断、评估和纠错的过程。**（它们的实现并不相同，也不能全部简单等同于 Agentic RAG。）
- **Agentic RAG**：有了 Function Calling、ReAct 和 Agent 运行时之后，检索就可以被包装成工具。Agent 接到的不再只是一个待搜索的 Query，而是一个**需要完成的目标。**它可以先规划，再调用向量库、关键词搜索、数据库、网页搜索等工具，根据中间结果继续调整。

9.3 Agentic RAG 到底要解决什么问题（五类）

很多人会说："我已经加了混合检索和 Rerank，为什么还需要 Agentic RAG？"**因为它们解决的不是同一层问题。**

> 混合检索和 Rerank 解决的是：**这一次怎么查得更准。**
> Agentic RAG 解决的是：**这次该不该查、该查几次、查什么、结果不够时下一步怎么办。**

#	要解决的问题	说明
1	**一个 Query 装不下复杂问题**	"对比两个版本并找出地区例外"至少包含版本检索、差异比较、地区过滤三个子问题。整句话直接向量化，可能每个方向都沾一点但哪个都查不准。Agent 可以先拆成子问题，分别找证据，再合并
2	**第一次检索错了，固定流程不会自救**	传统 RAG 一旦召回了不相关文档，后面的 Rerank、Prompt 和大模型都只能在**错误候选集**里挣扎。Agentic RAG 会先评估证据：相关性低、来源冲突或关键字段缺失，就重写 Query、扩大范围、换数据源，或明确告诉用户证据不足
3	**不同问题需要不同数据源**	政策解释适合查文档库，历史订单要查数据库，刚发生的变化可能要查网页或业务 API。**单一向量库不是企业全部知识**，Agent 的价值之一就是在多个受控工具间做路由
4	**不是所有问题都值得检索**	用户说"把上一段改得更口语化"，根本不需要查知识库。如果每次都跑 Query 改写、向量检索和 Rerank，只会增加延迟、Token 和费用。**Agent 可以判断直接回答，或走一条更轻的路径**
5	**多跳问题需要带着新线索继续查**	第一轮找到"新政策从 7 月生效"，第二轮才知道该查"7 月之后的订单"，第三轮再核对"华东区例外"。后一次检索依赖前一次结果，**很难靠一条预先写死的 Query 完成**

9.4 核心原理：五个角色 + 一个循环

底层并不神秘：把 Agent 的规划、工具、执行和反馈放进 RAG 在线链路。 一个实用的 Agentic RAG 通常有五个角色：

角色	职责
规划与路由	判断任务复杂度，决定直接回答、单次检索还是拆成多步
检索工具	向量检索、BM25、数据库、网页搜索和业务 API，各自暴露清晰能力
状态记录	保存子问题、已查来源、候选证据、冲突点、剩余预算
证据评估	检查相关性、完整性、来源可靠性和相互一致性
停止与生成	证据足够就组织带引用的回答；证据不足且预算耗尽就拒答或请用户补充信息

运行循环（ReAct 在检索场景的落地）：

```
理解目标
  ↓
决定：直接回答，还是调用检索工具？
  ↓
拆分子问题 → 选择数据源 → 执行检索
  ↓
评估证据：相关吗？完整吗？互相冲突吗？
  ↓
不够：改写 Query / 换工具 / 继续检索
足够：整合证据 / 标注来源 / 生成答案
```

> 模型基于当前状态决定行动，工具返回观察结果，模型再决定下一步。

重要提醒："反思"不能只靠模型拍脑袋说证据够了。 生产系统通常还要加**确定性规则**：至少需要几类证据、是否命中指定版本、引用能否回到原文、数值是否来自结构化数据、最大允许几轮、总耗时和费用上限是多少。

> **Agent 负责动态决策，规则负责兜底。两者缺一不可。**

9.5 用一个例子走完完整过程

用户问：对比 2025 和 2026 两版差旅报销政策，并说明上海分公司的特殊规定。

传统 RAG 的做法：把整句话检索一次，拿回几段最相似的内容就开始回答。如果召回的全是 2026 版，总结写得再流畅，也**没有完成"对比"**。

Agentic RAG 的做法：

1. 识别出三个**证据槽位**：2025 总部政策、2026 总部政策、上海分公司补充规定。
2. 为三个槽位分别生成 Query，**可以并行检索**，减少串行等待。
3. 检查**版本号、生效日期、适用组织和来源权限**，而不只看向量相似度。
4. 如果缺少 2025 版，就去历史归档库继续查；如果两个来源冲突，就优先核对正式发布记录。

5. 三类证据齐全后再做差异比较，并让**每条结论带回原始来源**。

> 你会发现，真正的变化不是"多查了几次"，而是**系统知道自己为什么查、还缺什么、什么时候算查完。**

9.6 Agentic RAG 不等于多 Agent

一个 Agent 配几个检索工具，加上明确的状态机和证据检查，**已经是 Agentic RAG**。

只有当任务确实存在稳定的职责边界（比如法规检索、财务数据查询和证据审计需要不同权限或不同模型时），才值得拆成多个 Agent。

> 否则，多 Agent 会带来更多**通信、上下文复制、错误传播和调试成本。先把单 Agent 的检索循环跑稳，再考虑多 Agent。**

9.7 它比传统 RAG 强，但也更难上线

维度	传统 RAG	Agentic RAG
路径	固定，容易预测	动态，每次可能不同
延迟	低且稳定	多轮工具调用，波动更大
成本	容易估算	与规划轮数、检索次数相关
调试	看固定链路	要看完整决策轨迹和工具结果
权限	一个检索入口较简单	多数据源需要逐工具鉴权
上限	适合简单、单跳问答	适合复杂、多跳、多源任务

落地时至少要守住六条：

1. **限制循环**：设置最大步骤、超时、Token 和工具调用预算，防止越查越远。
2. **工具要窄**：每个工具用途清晰，参数 Schema 和返回值稳定，别做一个"万能搜索"。
3. **权限前置**：检索必须继承用户权限，不能让 Agent 绕过原系统访问敏感数据。
4. **证据可回放**：记录每次 Query、命中文档、评分、选择理由和最终引用。
5. **失败能降级**：Agent 超时或判断不稳时，可以退回固定 RAG、澄清问题或拒答。
6. **分层评估**：既评估检索是否找全，也评估答案是否忠于证据；还要统计平均步骤、P95 延迟和单次费用。

9.8 什么场景值得用 Agentic RAG

适合的场景特征：

- 问题需要**多跳推理**，后一步依赖前一步找到的新线索；

- 一个请求包含**多个子问题**，需要拆解后分别检索；
- 知识分散在**文档库、数据库、网页和业务 API** 中；
- 对**证据完整性、来源引用和冲突处理**要求高；
- 用户问题**模糊**，需要先澄清意图或动态调整检索方向。

固定 RAG 往往更划算的场景：

- FAQ、产品手册等**简单单跳问答**；
- 数据源单一、Query 模式稳定、检索命中率已经很高；
- 对**首字延迟要求严格**，无法承受多轮模型和工具调用；
- 流程有明确规则，用 **Workflow** 就能可靠完成。

> **合理的升级顺序**：先把传统 RAG 的文档、切片、检索和评估做好，再增加路由与纠错，最后才开放更动态的 Agent 循环。
>
> **如果基础召回都做不好，套一层 Agent 只会让系统用更贵、更慢的方式找不到答案。**

9.9 面试里怎么解释 Agentic RAG

Q：Agentic RAG 和传统 RAG 有什么区别?

参考思路：

> 传统 RAG 的检索路径通常由开发者预先写死，用户问题经过一次或固定次数检索后直接生成。Agentic RAG 把检索封装成 Agent 工具，由模型根据任务状态决定是否检索、如何拆分 Query、选择哪个数据源，并对中间证据做评估；证据不足时可以改写或继续检索，足够时才停止并生成带引用的答案。它更适合复杂、多跳和多数据源问题，但会增加延迟、成本、可观测性和权限治理难度。

> 这段回答里，**定义、原理、场景和代价都有了**，比只说"Agentic RAG 更智能"有用得多。

9.10 本章关键结论

> **Agentic RAG 真正增加的不是一层 Agent 外壳，而是检索过程中的决策权、反馈和停止条件。**

参考资料（原文引用）： RAG 原始论文、ReAct、Self-RAG、Corrective RAG（CRAG）、Adaptive-RAG、AgenticRAG: Agentic Retrieval for Enterprise Knowledge Bases。

第 10 章 评估体系：检索质量 + 生成质量 + 线上采用率

10.1 为什么 RAG 必须有评估体系

很多团队做 RAG 优化是这样的：换个 Embedding 模型试试 → 感觉好点了 → 上线。**问题出在哪？**

后果	说明
无法量化优化效果	不知道这次改动让效果提升了 5% 还是 50%，也不知道在哪些场景提升了、哪些变差了
无法对比不同方案	A 方案用混合检索，B 方案用 Rerank，到底哪个更适合当前业务？没有评估就只能猜
无法发现退化	加了新功能后，老场景的效果有没有变差？没有评估就发现不了
面试答不上	面试官问"你们怎么验证优化有效"，只能说"感觉更好了"，这不是工程师的回答

> **真正的工程化 RAG，必须先建立评估体系，再做优化。这是应用开发和"调 API 玩玩"的本质区别。**

两层结构：

```
RAG 评估
├─ 检索侧评估（能不能找到）
│    ├─ Recall@K          相关内容有没有被找到？— 衡量覆盖率
│    ├─ Precision@K       找到的内容有多少是相关的？— 衡量纯度
│    ├─ MRR               最相关的结果排在第几位？— 衡量首位质量
│    └─ NDCG              整体排序质量如何？— 综合考虑位置和相关度
└─ 生成侧评估（用得好不好）
     ├─ Faithfulness      答案是否忠实于上下文，有无幻觉
     ├─ Answer Relevancy  答案是否真正回答了问题
     ├─ Context Relevancy 检索到的上下文对回答问题是否有用
     └─ Context Recall    生成答案所需关键信息是否都在上下文中
```

> 一句话：**检索侧保证能找到，生成侧保证用得好。没有评估的优化，就是拿着锤子找钉子。**

10.2 检索侧评估：四个核心指标

指标	定义	公式	工程含义
Recall@K	在 top-K 个检索结果中，正确答案片段出现的比例	`Recall@K = 检索到的相关 chunk 数 / 所有相关 chunk 总数`	**Recall 低（<70%）说明检索侧有严重问题**，相关内容根本没被找到，后面生成再好也没用。它是检索系统的**第一道防线**；Recall 不够，加 Rerank 也救不回来。优化方向：换 Embedding、调 Chunk 策略、引入混合检索、扩大 top-K
Precision@K	top-K 个检索结果中，相关内容占的比例	`Precision@K = 检索到的相关 chunk 数 / K`	**Precision 低说明噪声太多**，会稀释模型注意力。Precision 与 Recall 是一对矛盾：扩大 K 可提高 Recall 但会降低 Precision。优化方向：加 Rerank 精排、收紧检索阈值、改进相似度计算
MRR	第一个相关结果的排名倒数，在多个查询上取平均	`MRR = (1/N) * Σ(1/rank_i)`	衡量"**最相关的结果是否排在前面**"，对用户体验影响很大。搜索引擎、推荐系统常用，因为用户往往只看前几条
NDCG	考虑排序位置和相关度等级的**综合排序质量**指标	`DCG@K = Σ(rel_i / log2(i+1))`；`NDCG@K = DCG@K / IDCG@K`	最复杂也最全面：**相关度分级**（0-3 分而非相关/不相关）+ **位置折扣**（排前面的权重更高）+ **归一化**（与理想排序对比得 0-1 分）。适合**精细对比不同检索算法、评估 Rerank 效果**，计算成本高，通常用于离线评估

举例（同一场景贯穿四个指标）：

- 用户问"公司年假政策是什么？"，知识库里有 **3 个 chunk** 包含正确答案；
- 检索 top-5 结果里包含了其中 **2 个** → `Recall@5 = 2/3 = 66.7%`，`Precision@5 = 2/5 = 40%`；
- 3 个查询的第一个相关结果分别排在第 1、3、2 位 → `MRR = (1/1 + 1/3 + 1/2) / 3 = 0.611`。

关键对比：

对比	差别
Recall vs Precision	Recall 看"找没找到"；Precision 看"找到了多少垃圾"。`Recall@100 = 100%` 看似完美，但那 99% 的噪声会严重干扰生成质量
Recall vs MRR	Recall 关注"**有没有找全**"；MRR 关注"**最好的有没有排前面**"。两者都重要，侧重不同
实际工程平衡	**先用较大的 K（20~50）保证 Recall，再通过 Rerank 把 Precision 拉回来，最终只传 3~5 个高质量 chunk 给 LLM**
NDCG 用在哪	当你需要**精确对比两个检索方案的排序质量差异**时。日常监控更多用 Recall 和 MRR，因为标注成本低

10.3 生成侧评估：RAGAS 四个核心维度

检索侧保证找到了相关内容，但**最终答案质量如何，还需要生成侧评估**。难点在于：答案是自然语言，没有标准答案，很难用简单公式衡量。早期只能靠人工，成本高、不可扩展。

RAGAS（Retrieval Augmented Generation Assessment） 提出了一套基于 LLM 的自动化评估方法，目前已成为 RAG 生成评估的**事实标准**。

维度	定义	评估方法	工程意义与优化方向
Faithfulness（忠实度）	生成的答案是否**忠实于检索到的上下文**，有没有编造事实	把答案拆成多个陈述（statements）；用 LLM 判断每个陈述能否从上下文推导；`Faithfulness = 可推导陈述数 / 总陈述数`	衡量 RAG 是否真正 **Grounded（有根据的）**，这是 RAG 相比纯 LLM 的核心优势。低分说明模型在自由发挥。优化：改进 Prompt 约束（"请严格根据以下资料回答"）、使用更强模型、减少上下文噪声
Answer Relevancy（答案相关性）	生成的答案是否**真正回答了用户的问题**，有没有答非所问	用 LLM 根据生成的答案**反向生成 N 个可能的问题**；计算这 N 个问题与原始问题的语义相似度；`Answer Relevancy = (1/N) * Σ similarity(original_query, generated_query_i)`	衡量答案是否**切题**，防止"相关但不精准"。低分说明模型理解问题有偏差，或上下文干扰了判断。优化：优化 Prompt 明确回答要求、改进 Query 理解、减少无关上下文
Context Relevancy（上下文相关性）	检索到的上下文中，**有多少内容对回答问题是真正有用的**	用 LLM 从每个 chunk 中提取与问题相关的句子；`Context Relevancy = 相关句子数 / 总句子数`	衡量**检索质量和 Chunking 质量**，是检索侧评估的补充（从生成角度反向评估检索）。优化：改进 Chunking 策略（chunk 太大会包含太多无关内容）、加 Rerank、减少 top-K
Context Recall（上下文召回）	**ground truth 答案中的关键信息，有多少出现在检索到的上下文中**	需要标注标准答案；用 LLM 判断标准答案中每个陈述能否从上下文推导；`Context Recall = 可从上下文推导的陈述数 / 标准答案总陈述数`	从生成角度评估**检索完整性**。低分说明检索漏掉关键信息，生成效果再好也无法给出完整答案。优化：扩大 top-K、改进检索算法、检查索引覆盖

逐项示例（便于理解）：

Faithfulness

- 上下文："公司年假为 10 天，工龄满 3 年增加到 15 天。"
- 生成答案："公司年假为 10 天，满 3 年可增加到 15 天，**满 5 年可增加到 20 天**。"

- 前两个陈述可从上下文推导，第三个是编造的 → `Faithfulness = 2/3 = 66.7%`

Answer Relevancy

- 原始问题："公司年假政策是什么？"
- 生成答案："公司实行弹性工作制，员工可以远程办公。年假为 10 天。"
- LLM 反向生成的问题可能是"公司工作制度是什么？"，与原问题相似度低 → `Answer Relevancy 低`，说明答案偏题（虽提到年假，但重点跑偏）

Context Relevancy

- 问题："公司年假政策？"
- 检索到的 chunk："公司成立于 2010 年，主营业务是软件开发。员工福利包括五险一金、**年假 10 天**、节日礼品。公司地址在北京朝阳区。"
- 只有"年假 10 天"这一句相关 → `Context Relevancy = 1/4 = 25%`

Context Recall

- 问题："公司年假政策？"
- 标准答案："基础年假 10 天，工龄满 3 年增加到 15 天。"（2 个陈述）
- 检索上下文只提到"基础年假 10 天" → `Context Recall = 1/2 = 50%`

10.4 RAGAS 四个指标的关系

```
Context Recall + Context Relevancy  =  检索质量（从生成角度看）
Faithfulness     + Answer Relevancy   =  生成质量
```

理想情况：

- Context Recall 高 → 检索找全了关键信息
- Context Relevancy 高 → 检索结果干净、噪声少
- Faithfulness 高 → 模型忠实基于检索结果回答
- Answer Relevancy 高 → 答案切题，没有跑偏

因果链（重要）：

> 如果 **Context Recall 低**，后面其他指标再高也没用，因为**关键信息根本没检索到。**
>
> 如果 **Context Relevancy 低**，会**拉低 Faithfulness 和 Answer Relevancy**，因为噪声太多干扰了生成。

10.5 如何构建 RAG 评估集

评估集需要包含：**问题（Query）** + **标准答案（Ground Truth，可选，用于 Context Recall）** + **相关文档片段（Relevant Chunks，用于检索侧 Recall/Precision）**。

三种构建方法：

方法	做法	质量	成本	适用
① 人工构造	挑选业务高频问题，人工标注正确答案和相关 chunk	最高	最高	核心业务场景，通常 **50~200 条**即可建立 baseline
② 真实日志采样	从生产日志采样真实用户问题，人工标注答案质量（好/中/差）	中等	中等	**持续监控**，发现线上问题
③ LLM 生成	用 LLM 根据知识库内容生成问题-答案对	较低	最低	快速验证；注意 LLM 生成的问题可能**过于理想化**，不代表真实用户行为

工程实践建议： 先用方法 ③ 快速搭建评估框架 → 再用方法 ① 构建核心场景的高质量评估集（50~100 条）→ 用方法 ② 持续补充，逐步扩大评估覆盖。

10.6 离线评估 vs 在线评估

	离线评估	在线评估
做法	在测试集上跑指标，优化前后对比	真实用户使用时收集反馈
优点	可控、可重复、成本低	反映真实效果
缺点	测试集可能不代表真实场景	反馈稀疏、噪声大
信号	指标数值	**显式**：点赞/点踩、提交纠错；**隐式**：是否继续追问、是否复制答案、停留时长

> 工程实践：**用离线评估指导优化方向，用在线评估验证真实效果。离线评估是必要条件，在线评估是充分条件。**

10.6.1 线上采用率：用户真的用了你的答案吗

离线指标再漂亮，如果用户不采纳答案，就是白搭。采用率（Adoption Rate）是 RAG 系统最真实的质量信号。

信号类型	指标	含义
直接采用（强信号）	复制率	用户复制了答案内容，说明有 value（复制率 = 复制次数 / 总回答次数）
	点赞率	用户主动点赞
	引用率	用户在后续对话或文档中引用了这个答案
间接采用（弱信号但规模大）	无追问率	看完答案后没有继续追问，可能说明问题解决了（**也可能是放弃了**，需结合停留时长）
	停留时长	3~10 秒可能是扫一眼就关闭；30 秒以上可能是认真阅读
	跳出率	是否立即关闭页面或切换到其他工具
负向信号（问题指标）	重复提问率	换个问法再问一遍，说明上次答案没解决问题
	点踩率	用户主动给差评
	人工接管率	用户选择转人工客服，说明 RAG 答不上来

采用率监控示例（每天 1000 个问题）：

```
复制率   15%  → 150 个答案被复制（强信号）
点赞率    8%  →  80 个点赞
点踩率    3%  →  30 个差评
无追问率 60%  → 600 个问题一次解决

综合采用率 = (复制次数 + 点赞次数×2 - 点踩次数×2) / 总回答次数
```

> **为什么点赞/点踩权重 ×2？** 因为用户主动点击的成本比复制高，信号更强。

最常见的坑（必读）：离线指标涨了，采用率反而降了。

这是很多团队踩过的坑：离线 `Recall@5` 从 70% 优化到 85%，RAGAS `Faithfulness` 从 0.75 提到 0.88，结果上线后**采用率反而下降**。可能原因：

- **过度优化了测试集**：测试集的问题分布和真实用户不一致；
- **延迟变长了**：加了 Rerank 和多轮检索，P95 延迟从 2 秒涨到 5 秒，用户等不及关了；
- **答案变长了**：Faithfulness 高是因为把检索到的内容都塞进答案，用户看不完；
- **语气变硬了**：为提高 Faithfulness，Prompt 约束太强，答案变成"根据资料显示……"的八股文，用户不爱看。

> **所以线上采用率是最终裁判。离线指标是过程指标，采用率是结果指标。**

10.6.2 评估的延迟代价：快和准怎么平衡

评估不是免费的，每个评估环节都有**延迟成本**。

RAGAS 的延迟来源（四个指标都需调用 LLM 做判断）：

指标	大致 LLM 调用次数
Faithfulness	2~3 次（拆分陈述、逐条判断）
Answer Relevancy	1~2 次（反向生成问题、计算相似度）
Context Relevancy	= chunk 数量（每个 chunk 提取相关句子）
Context Recall	1~2 次（判断标准答案的每个陈述）

> 假设每次 LLM 调用 200ms，评估一个答案可能额外需要 **1~2 秒**。线上实时评估会显著增加用户等待时间。

成本代价：用户问答 = 输入 500 + 输出 200 = 700 tokens；RAGAS 每个指标可能需要 1000~2000 tokens，四个指标下来，**评估 Token 可能是原问答的 4~8 倍**。

大多数生产系统的做法：

- **线上不做完整评估**：只收集原始数据（问题、检索结果、答案、用户反馈）；
- **离线批量跑评估**：每天/每周从线上采样 1000~5000 条，跑完整 RAGAS；
- **线上只做轻量监控**：计算简单指标（答案长度、检索延迟、用户反馈率）。

分级评估策略（推荐）：

风险等级	占比	评估方式	代价
高风险查询（涉及金额、法律条款）	5%	实时 RAGAS 全量评估	延迟 +2 秒
中风险查询	20%	实时 Faithfulness 单项评估	延迟 +500ms
低风险查询	75%	仅记录日志，离线评估	无额外延迟

面试话术（可直接用）：

> "我们根据查询风险分级。高风险查询（涉及金额、法律条款）实时跑 Faithfulness 检查，宁可多花 500ms 也要防幻觉；中低风险查询只记录日志，每天离线采样评估，发现问题后针对性优化。这样既保证了核心场景质量，P95 延迟也控制在 2 秒以内，成本只增加了 15%。"

> **有数据、有分级、有权衡，这就是工程化的回答。**

10.7 评估驱动的优化闭环（五步）

```
1. 建立 Baseline —— 在评估集上跑当前系统，记录所有指标作为基准
        ↓
2. 定位瓶颈 —— Recall 低？Precision 低？Faithfulness 低？针对性分析
        ↓
3. 针对性优化 —— 每次只改一个变量（换模型 / 加 Rerank / 调 Chunk）
        ↓
4. 评估验证 —— 在评估集上重新跑指标，与 baseline 对比
        ↓ 效果提升？
     是 → 采纳 / 继续迭代        否 → 回滚或继续调整
        ↓
5. 持续迭代 —— 新问题加入评估集，定期重新评估，监控线上
```

瓶颈定位速查表：

症状	优先排查
Recall@5 < 70%	优先优化**检索召回**
Recall 高但 Precision 低	加 **Rerank**
Context Relevancy 低	优化 **Chunking** 或减少 **top-K**
Faithfulness 低	改进 **Prompt 约束**
Answer Relevancy 低	检查 **Query 理解**和**模型能力**

> **核心：建立可量化的评估体系 → 每次只改一个变量 → 数据驱动决策 → 形成优化闭环。**
>
> **错误做法：凭感觉优化 → 同时改多个地方 → 不知道哪个起作用 → 无法复现效果。**

10.8 知识拓展 Q&A

Q1：检索评估一定要人工标注相关 chunk 吗？不一定。如果有标准答案，可以用 LLM 判断"标准答案能否从某个 chunk 推导出来"，自动化标注相关性。但**核心场景建议人工标注一部分作为"黄金标准"**。

Q2：RAGAS 评估依赖 LLM 判断，会不会不准？确实可能有偏差。工程实践：在核心评估集上**对比 LLM 评估和人工评估的一致性**，如果一致性高（比如 85% 以上），就可以信任 LLM 评估做大规模自动化；如果一致性低，说明评估 Prompt 需要调整，或该场景确实需要人工介入。

Q3：Recall 和 Precision 哪个更重要？看业务场景。**答不上来的代价高**（医疗、法律咨询）→ Recall 更重要，宁可多召回也不漏关键信息；**噪声的代价高**（客服机器人给错答案会被投诉）→ Precision 更重要，宁可少答也不能答错。**通常做法：用大 K 保 Recall，用 Rerank 保 Precision，两头都要。**

Q4：评估集要多大才够？取决于业务复杂度。简单场景（FAQ 类）50~100 条可建立可信 baseline；复杂场景（开放域问答）可能需要 300~500 条覆盖主要 case。**关键是评估集要有代表性，而不是一味追求数量。**

先从核心高频场景开始，逐步扩展。

Q5：怎么在面试中快速讲清 RAG 评估？按三层结构：**检索侧指标（Recall/Precision/MRR）+ 生成侧指标（RAGAS 四维度）+ 评估驱动优化闭环。**先说清评估分两侧、每侧核心看什么；再说怎么建评估集、怎么形成优化闭环。最后举个例子：

> "我们发现 Recall@5 只有 65%，优化 Embedding 模型后提升到 82%，再加 Rerank 后 Precision 从 30% 提升到 70%，最终答案的 Faithfulness 从 0.75 提升到 0.88。"

——**有框架、有数据、有闭环，这就是好回答。**

Q6：线上采用率低但离线指标高，问题出在哪？三个最常见原因：① **测试集偏差**（离线测试集都是"标准问题"，真实用户问法千奇百怪、有错别字、有歧义）；② **延迟变长**（优化提升了质量，但 Rerank、多轮检索让延迟从 2 秒涨到 5 秒）；③ **答案不符合阅读习惯**（为提高 Faithfulness，答案变成"根据资料第 3 条显示……"的机器腔）。

> 诊断方法：**从采用率最低的 100 条 case 里随机抽 20 条，自己当用户体验一遍**，看看是答案质量问题还是交互体验问题。

Q7：评估成本太高怎么办？

- **采样评估**：每天随机采样 1%~5% 跑完整评估；
- **分级评估**：核心场景全量评，边缘场景抽样评；
- **单项评估**：根据当前优化目标只跑 1~2 个（如优化检索时只看 Context Recall 和 Context Relevancy）；
- **换便宜模型做评估**：Faithfulness 等判断任务用 GPT-4o-mini 或 Claude-Haiku，**成本降 10 倍，准确率只降 5%。**

Q8：开源工具怎么选？

- **检索评估**：自己写脚本计算 Recall/Precision/MRR 比较简单，也可用 `ranx` 库；
- **RAGAS 评估**：直接用 `ragas` 库，已实现四大指标的 LLM 评估逻辑，支持多种 LLM 后端；
- **评估集管理**：可用 LangSmith 或 Weights & Biases 做实验跟踪和对比。

> **没有完美的评估，只有持续改进的评估。** 从简单指标开始，逐步建立适合自己业务的评估体系，比一上来就想做完美评估更现实。

10.9 本章关键结论

> **离线指标是方向盘，线上采用率是目的地。两个都要看，但别忘了用户才是最终裁判。**

第 11 章 面试应答框架：高频问题与回答结构

11.1 面试官真正想听什么

典型错误回答："我们用了向量数据库，还加了混合检索和 Rerank，效果提升挺明显。"

方向不算错，但有三个硬伤：

- 没说**原来出了什么问题**，为什么要优化；
- 没说**为什么选这些方案**，付出了什么代价；
- **没有基线和指标**，"明显"只是主观感觉。

> **面试官真正想听的不是技术名词，而是你有没有完整做过"发现问题、做出取舍、验证结果"这条工程链路。**

11.2 五个落点（回答任何 RAG 问题的骨架）

落点	要讲清什么
① **业务目标**	知识从哪来，为什么需要 RAG，不用 RAG 会出什么问题
② **完整链路**	离线怎么建库，在线怎么检索、重排、拼接和生成
③ **选型取舍**	Chunk、Embedding、向量库、Top-K、混合检索为什么这么选
④ **问题诊断**	答案不准时，先区分检索没找到，还是模型没用好
⑤ **指标证据**	用检索、生成、线上业务三层指标证明方案有效

> 回答单个问题时按此顺序组织：**先给结论，再讲项目事实，然后说明取舍，最后给指标和边界。**

11.3 面试官会沿着项目一直往下挖（追问阶梯）

面试官通常不会随机抽十个名词让你背。他会从简历上一句"负责企业知识库 RAG 系统"开始，顺着同一条项目链不断追问：

```
简历：做过 RAG 项目
  ↓ 业务：为什么需要 RAG？
  ↓ 链路：离线与在线怎么串？
  ↓ 选型：参数和方案为什么这样定？
  ↓ 排障：答案不准先查哪一侧？
  ↓ 证据：哪些指标证明有效？
  ↓ （更深层：索引源码 · Embedding 训练 · 分布式实现）
```

应用开发者必答线：链路、选型、排障、指标。

底层细节（索引算法源码、Embedding 训练目标等）：知道原理和工程影响即可，**不要硬装成算法专家。**

真正拉开差距的不是背到多底层，而是必答区里的每一层都有项目证据。

11.4 通用回答框架（五步）

很多人上来先报技术栈："LangChain、Milvus、BGE、Rerank 全用了。"——**这相当于把工具箱倒在桌上，却没说你修的是什么问题。**

更稳的回答顺序：

步骤	内容
① **结论**	先直接回答面试官的问题
② **项目事实**	说清数据、场景、症状或约束
③ **方案取舍**	为什么选 A 不选 B，代价是什么
④ **指标证据**	基线和改后分别怎样
⑤ **适用边界**	什么情况下这套方案不成立

示例：面试官问"为什么加 Rerank？"

❌ 不要只说："Rerank 能提高准确率。"

✅ 可以这样答：

"我们先看评估集，发现相关 Chunk 多数已经进入 Top-20，但真正能回答问题的片段经常排不到前 5，所以问题不在召回覆盖，而在候选排序。我们保留前面的快速召回，再用 Rerank 做精排，只把 Top-N 交给模型。代价是多了一段推理延迟，所以候选数和 Top-N 是根据离线排序指标、端到端答案质量和 P95 延迟一起定的。"

这段话里有**症状、有判断、有方案，也有代价。**即使暂时不报具体数字，也比"Rerank 更准"有说服力。

11.5 八个高频问题怎么答

Q1：为什么这个项目要用 RAG?

踩雷答法："因为大模型有幻觉，所以我们用了 RAG。"——太泛。大模型有幻觉，不代表所有场景都要建知识库。

回答框架：业务缺口 → RAG 能力 → 不选其他方案的原因。

示范：

> "这个场景依赖企业内部文档，知识会更新，而且答案需要展示来源。基础模型既接触不到私有数据，也不能保证引用可追溯，所以我们用 RAG 在生成前动态检索证据。动态知识不适合直接训进参数；如果只是固定格式或语气问题，我会先用 Prompt，必要时才考虑微调。"

> 要说清楚：**RAG 解决的是外部知识接入，不是给所有模型问题贴创可贴。**

Q2：从用户提问到最终回答，完整链路是什么?

踩雷答法："文档向量化，用户提问后做相似度检索，再让大模型回答。"——只说出三个大词，离真实链路还差一截。

回答框架：离线建库 + 在线问答 + 质量控制。

可直接复述的话术：

> "离线侧先解析和清洗文档，按结构切成 Chunk，补充来源、时间、权限等元数据，再用 Embedding 模型向量化并建立索引。在线侧先处理 Query，经过向量检索或混合检索召回候选，再按需要做 Rerank，筛出高质量上下文，带着来源信息拼进 Prompt，最后由模型生成答案。**生成后还要做引用、无答案拒答、权限和日志监控。**"

Q3：Chunk 多大？Overlap 多少？为什么?

踩雷答法："Chunk 设 500，Overlap 设 100，这是比较常见的配置。"——参数可能没错，但"常见"不是选型依据。面试官再问一句"为什么不是 300"，回答就断了。

回答框架：文档结构 → 用户问题粒度 → 参数候选 → 评估结果。

示范：

> "Chunk 大小不是固定值。我们先看文档结构和问题粒度：FAQ 按问答对切，Markdown 优先保留标题层级，合同或制度文档尽量不拆散完整条款。Chunk 过小会丢上下文，过大会混入多个主题。Overlap 只用来保护边界信息，不是越大越好；过大会造成重复召回和上下文浪费。最终会在业务评估集上对比不同参数的 Recall@K、排序质量和答案忠实度。"

> **好回答不是报出一个神奇数字，而是说清数字怎么来的。**

Q4：Embedding 模型和向量数据库怎么选？

这道题最容易答成**产品点名大会。Embedding 看的是检索效果和部署约束，向量数据库看的是数据规模与工程能力。两者不是一回事。**

Embedding 选型至少讲四个维度：

- 业务语言和垂直领域是否匹配；
- 在自有评估集上的 Recall@K、MRR 或 NDCG；
- 向量维度带来的存储和检索成本；
- 本地部署、调用延迟、吞吐和数据合规。

向量数据库选型至少讲五个维度：

- 数据量和并发量；
- 索引类型及查询延迟；
- 持久化、扩缩容和高可用；
- 元数据过滤、增量更新和多租户能力；
- 团队运维成本。

收口话术：

> "我不会因为某个模型榜单高就直接选，也不会因为某个向量库流行就上。先用业务评估集验证召回效果，再结合延迟、成本、合规和运维约束做选择。"

Q5：为什么要混合检索？为什么还要 Rerank？

先分清两者解决的问题：混合检索补召回，Rerank 提排序。

- 向量检索擅长语义相似，但产品型号、错误码、法规编号、人名等**精确词**可能漏掉；
- BM25 擅长关键词命中，但**不理解改写和近义表达**；
- 两路召回再融合 → 让**候选池覆盖更完整。**
- Rerank 发生在候选池之后，用更精细的相关性模型重新排序，解决的是"**找到了，但真正有用的 Chunk 排得不够靠前**"。

回答时还要主动说代价：

- 混合检索增加一套检索与融合链路；
- Rerank 增加推理延迟和算力成本；
- **Recall@K 本来就低时，Rerank 救不回根本没召回的内容**；
- 简单、低风险场景不一定值得把整套链路都加上。

> 这比"生产环境标配混合检索加 Rerank"更靠谱。**工程里没有无条件的标配，只有指标和约束下的选择。**

Q6：RAG 答不准，你怎么排查?

这是 RAG 面试里**最能区分"跑过 Demo"和"做过工程"**的问题。**第一步不要换模型，也不要立刻加 Rerank。先看 Top-K 候选里有没有支撑正确答案的 Chunk。**

检查结果	排查方向
没有	优先查**文档解析、索引覆盖、Chunk、Embedding、Query 和召回策略**
有，但排序靠后	查**融合、Rerank、Top-K 和上下文噪声**
有且位置合理，答案仍然错	查 **Prompt 拼接、引用约束、上下文冲突和模型能力**
知识库本来就没有	应该**拒答或转人工**，不能逼模型编一个答案

> **先分流，再优化；一次只改一个变量。**

Q7：你们做了哪些优化?

踩雷答法："做了 Query 改写、混合检索、Rerank、父子块和 Context 压缩。"——全都做不代表做得好。一次改五个变量，最后甚至不知道是谁起了作用。

更好的回答：

> "我们先建立 Baseline，把失败样本按检索侧和生成侧分类。评估发现主要问题是……，所以先改……。每次只调整一个变量，用同一评估集比较改前改后，再观察端到端质量、P95 延迟和单次成本。达到收益门槛才保留，否则回滚。"

> **回答里最好只选一两个你真正做过的优化，讲透四件事**：优化前的症状 / 如何定位根因 / 为什么选择这个方案 / 哪些指标发生了变化。

Q8：怎么证明 RAG 真的有效?

指标要分三层讲。

① **检索侧**（有没有找到、排得好不好）：

- Recall@K 看相关证据是否进入候选；
- Precision@K 看候选里有多少是相关内容；
- MRR 看第一个相关结果排得够不够靠前；
- NDCG 看多级相关性下的整体排序质量。

② **生成侧**（模型有没有把证据用好）：

- 忠实度看答案是否被上下文支持；
- 答案相关性看有没有正面回答问题；
- 引用正确率看来源能否支撑对应结论；
- 无答案场景看**正确拒答率**。

③ **线上业务侧**（系统是否真的可用）：

- 任务成功率、用户反馈或转人工率；
- P95/P99 延迟、超时率；
- 单次请求成本、每个成功任务成本；
- 越权检索、敏感信息泄露等风险指标。

面试句式模板：

> "优化前，评估集上的 **[指标]** 是 **[基线]**；定位到 **[根因]** 后，我们做了 **[单一改动]**，该指标变成 **[结果]**。同时 **[延迟/成本/风险指标]** 变化了 **[真实数据]**，所以最终选择 **[上线、灰度或放弃]**。"

> **方括号必须换成你项目的真实数据。** 忘了精确数值可以说明统计口径和大致区间，**不能现场编一个漂亮结果。**

11.6 面试里最容易暴露的四个问题

问题	改法
只报技术栈，不讲问题	每个工具后面补一句"它解决了什么问题，为什么适合当前约束"
把参数说成行业标准	讲候选范围、对比实验和最终指标，不要把个人配置包装成通用答案
只讲准确率，不讲成本和延迟	质量达标后，再比较端到端延迟和每个成功任务成本
编指标	没有线上规模就如实说是离线评估；没有自动指标就说明人工抽检规则；没记住数值就讲方法和区间

11.7 如果只做过 Demo，怎么答？

不要硬说自己做过生产级 RAG。

可以明确说明：

> "这个项目目前做到离线 Demo，没有真实线上流量，所以我验证的是检索和生成质量，没有声称做过高并发生产验证。"

然后继续讲：

- 你如何构造测试问题和标准证据；
- 你对比过哪些方案；
- 失败样本暴露了什么问题；
- 如果上线，还要补哪些监控、安全和容量工作。

> **诚实地讲清边界，比虚构一个生产项目更像工程师。**

11.8 知识拓展 Q&A

Q1：Top-K 是不是越大越好？不是。K 增大通常能提高召回覆盖，但也会带来更多噪声、Token、延迟和生成干扰。**先看 Recall@K 曲线，再结合 Rerank 后的上下文质量和端到端指标选**，不要只追求召回。

Q2：有了 Rerank，还需要优化 Embedding 吗？需要。**Rerank 只能重排已经召回的候选。**相关内容根本没进候选池，它就无能为力。**先保证召回，再优化排序。**

Q3：RAG 能彻底消除幻觉吗？不能。检索可能找错，文档本身可能过期，模型也可能忽略或曲解证据。**RAG 降低了无依据生成的概率**，但仍要配合引用、拒答、权限、时效性和人工兜底。

Q4：向量数据库能不能直接用普通数据库代替？要看规模和能力需求。数据量小、数据库本身支持向量类型和近似检索时，可以先用现有数据库降低复杂度；当向量规模、查询吞吐、索引能力、过滤和扩缩容要求上来后，再评估独立向量数据库。

Q5：面试官追问没负责的环节怎么办？直接说明责任边界，再讲你理解的接口和影响。例如：

> "索引集群由基础设施同学维护，我负责检索策略和评估；我关注索引参数对召回、延迟和内存的影响，但没有改过底层实现。"

> 这比把团队成果全揽到自己身上更可信。

11.9 本章关键结论

> **别再把 RAG 面试答成工具名接龙。把问题怎么发现、方案为什么选、收益怎么验证讲清楚，面试官自然知道你是不是做过。**

附录 A：核心公式速查

指标	公式
Recall@K	`检索到的相关 chunk 数 / 所有相关 chunk 总数`
Precision@K	`检索到的相关 chunk 数 / K`
MRR	`(1/N) * Σ(1/rank_i)`
DCG@K	`Σ(rel_i / log2(i+1))`
NDCG@K	`DCG@K / IDCG@K`
Faithfulness	`可推导陈述数 / 总陈述数`
Answer Relevancy	`(1/N) * Σ similarity(original_query, generated_query_i)`
Context Relevancy	`相关句子数 / 总句子数`
Context Recall	`可从上下文推导的陈述数 / 标准答案总陈述数`
综合采用率	`(复制次数 + 点赞次数×2 - 点踩次数×2) / 总回答次数`
RRF 融合分	对各路结果取 `1/rank` 求和后重排

附录 B：指标体系速查表

层次	指标	回答什么问题	典型红线
检索侧	Recall@K	相关内容有没有被找到	**< 70% 说明检索侧有明显问题**
检索侧	Precision@K	找到的内容有多少是相关的	过低说明噪声多，需加 Rerank
检索侧	MRR	最相关的结果排在第几位	低说明最好的结果没排前面
检索侧	NDCG	整体排序质量如何	用于精细方案对比
生成侧	Faithfulness	答案有没有幻觉	低说明未严格基于资料
生成侧	Answer Relevancy	答案有没有答非所问	低说明理解问题有偏差
生成侧	Context Relevancy	上下文有没有用	低说明 Chunking 或 top-K 有问题
生成侧	Context Recall	关键信息有没有都检索到	**低则后面指标再高也没用**
长文档/代码	父级补全正确率	命中 Child 后是否拿到正确父节点	—
长文档/代码	版本错误率	是否引用了目标 commit 之外的内容	目标为 0
长文档/代码	越权召回率	无权内容进入候选的比例	**目标必须是 0**
长文档/代码	Context 有效率	进入上下文的 Token 有多少真正支持答案	—
长文档/代码	索引新鲜度	代码更新到可检索之间的延迟	—
线上业务	复制率/点赞率/点踩率	用户是否真的采纳答案	—
线上业务	P95/P99 延迟、超时率	系统是否够快够稳	—
线上业务	单次请求成本	是否可规模化	—
Agentic RAG	平均步骤数、单次费用	决策是否失控	需设预算上限

附录 C：参数经验值速查

参数	经验值	备注
Chunk size（对话型问答）	256 ~ 512 token	无通用最优值，需按业务调
Chunk size（知识库检索）	512 ~ 1024 token	同上
Chunk size（长文档摘要）	1500 ~ 2000 token	同上
Overlap	chunk 大小的 **10% ~ 20%**	不是越大越好，过大会重复召回
向量维度	384（轻量）/ **768（均衡起点）** / 1536（高精度）	—
top-K（召回）	3 ~ 10（基础）；工程上常用 **20 ~ 50**	大 K 保 Recall，再用 Rerank 拉 Precision
Rerank 候选规模	top-20 以内	Cross-Encoder 逐对推理，规模受限
最终传入 Context	**3 ~ 5 条**高质量 chunk	宁少勿滥
HNSW 超参数	`M`（每节点最大连接数）、`efConstruction`（建图搜索深度）	越大精度越高，内存与构建时间成比例增加
高风险查询实时评估占比	5%	延迟 +2 秒可接受
中风险查询单项评估占比	20%	延迟 +500ms
低风险查询	75%	仅记录日志，无额外延迟
LLM 评估采样率	每天 1% ~ 5%	成本控制
评估集规模	FAQ 类 50~100 条；开放域 300~500 条	关键是有代表性

附录 D：高频面试题速答卡（30 问）

#	问题	一句话答案
1	为什么有了大模型还需要 RAG?	大模型有幻觉、私有知识盲区、时效性、无法溯源四大局限，RAG 通过在生成前检索外部知识把相关片段注入上下文，针对性缓解
2	RAG 能解决幻觉吗?	能显著缓解不能根治，它降低无依据生成的概率，但模型仍可能对检索结果做错误推理
3	什么时候不用 RAG?	创意生成、强推理任务、模型已良好覆盖的稳定领域，此时只增加延迟和成本
4	完整 RAG 链路?	离线：加载→清洗→切片→向量化→入库（含元数据）；在线：Query→（改写）→向量化→召回 top-K→（混合检索/Rerank）→拼 Context→生成→带引用答案
5	Embedding 离线在线必须一致吗?	必须，不同模型向量空间不同，混用会让相似度计算完全失效
6	chunk 怎么切?	有结构用结构感知，普通散文用递归字符（默认），质量要求极高用语义切分，原型用固定长度
7	chunk 太大太小的问题?	太小语义不完整、精度低、索引膨胀；太大话题混杂、向量平均化、噪声稀释注意力
8	为什么要 overlap?	防止重要信息落在 chunk 边界被割裂，建议 chunk 大小的 10%~20%
9	Embedding 是什么?	把文本映射到高维向量空间，用空间距离表达语义相似度，本质是**压缩语义**而不是生成文本
10	Embedding 怎么选?	看语言场景、最大 token 限制、向量维度、是否支持非对称检索；中文优先 BGE/M3E
11	Embedding 和 Rerank 区别?	Embedding 双塔独立编码、可预计算、快、粗筛；Rerank 交叉编码器联合建模、不可预计算、慢、精排
12	换 Embedding 模型要注意什么?	必须全量重建索引，否则新旧向量不可比
13	为什么不用 MySQL 做向量检索?	功能上 B-Tree 无法表达高维几何相似度；性能上全量计算是 O(N)，向量库用 ANN 降到近似 O(log N)
14	ANN 是什么?	近似最近邻搜索，通过 HNSW/IVF 等索引只访问少量向量，精度 95%+，速度提升数量级
15	HNSW 和 IVF 的区别?	HNSW 分层图，速度快精度高但内存高；IVF 聚类分桶，内存低适合超大规模但有边界效应
16	向量库选型?	几十万以内 Chroma/pgvector；百万以上需过滤与混合检索用 Milvus/Qdrant；云托管免运维用 Pinecone
17	FAISS 是向量数据库吗?	不是，它是纯索引库，无持久化、无元数据管理、无 CRUD

#	问题	一句话答案
18	RAG 答不准怎么排查?	先分流：手动看 top-K 有没有正确答案——有是生成侧问题，没有是检索侧问题；再用 Recall@K 量化
19	检索侧常见问题?	文档质量差、chunk 粒度不合理、Embedding 选型不当、召回不足、缺少 Rerank
20	生成侧常见问题?	Prompt 拼接有问题、上下文干扰、缺少引用约束、模型理解能力弱
21	做了哪些优化?	按 Query 侧（改写/Multi-Query）、检索侧（混合检索/元数据过滤）、精排侧（Rerank）、Context 侧（父子块/压缩）分类，配症状—方案—指标
22	为什么要混合检索?	向量擅长语义但对精确词弱，BM25 擅长精确匹配但不理解近义；两路融合让候选池更完整，用 RRF 融合
23	什么时候该加 Rerank?	Recall@K 已足够好但答案精准度仍不高时；Recall 本身就低时应先解决召回
24	父子块检索解决什么?	小 chunk 检索保证精度，返回大 chunk 保证上下文完整，兼顾两者
25	长文档和代码检索的三个硬问题?	结构不能断、版本不能错、权限不能漏；关键是检索单位≠返回单位
26	代码为什么不能按字符切?	代码自然边界是模块/类/函数/语句块，应按 AST 节点切，并保留符号名、路径、分支、commit
27	Agentic RAG 和传统 RAG 区别?	传统把检索路径写死，Agentic 把检索变成 Agent 按需使用的工具，由模型决定是否检索、怎么拆、查几次、何时停止
28	Agentic RAG 要守哪些底线?	限制循环、工具要窄、权限前置、证据可回放、失败能降级、分层评估
29	RAG 怎么评估?	检索侧 Recall/Precision/MRR/NDCG，生成侧 RAGAS 四维度（Faithfulness/Answer Relevancy/Context Relevancy/Context Recall），再加线上采用率
30	离线指标好但线上采用率低?	测试集偏差、延迟变长、答案不符合阅读习惯；从采用率最低的 100 条抽 20 条亲自体验来诊断

附录 E：常见误区总清单

认知类

- ❌ RAG = 向量检索 → 向量检索只是在线侧的一个步骤
- ❌ RAG 能彻底消除幻觉 → 只降低无依据生成的概率
- ❌ 有了 Rerank 就不用管 Embedding → Rerank 只能重排已召回的候选
- ❌ RAG 适合所有大模型应用 → 创意生成、强推理、训练数据已覆盖的领域不适合
- ❌ 用 FAISS 就是用了向量数据库 → FAISS 只是索引库

选型类

- ❌ Embedding 模型越大越好 → 轻量但针对检索优化的模型可能更合适
- ❌ 直接用 OpenAI Embedding 就行 → 中文场景建议评测对比，BGE/M3E 常更优
- ❌ 换 Embedding 不用重建索引 → **完全错误**，必须全量重建
- ❌ Embedding 和 Rerank 用完全不同的模型没关系 → 训练数据分布差异会让评分口径不一致
- ❌ 向量库参数越高越好 → 精度/速度/内存需要平衡
- ❌ 向量相似度高就等于语义相关 → 取决于底层 Embedding 模型质量

工程类

- ❌ 只要模型够强，Chunking 随便切 → 模型能力无法弥补检索质量的缺陷
- ❌ Rerank 一定要加 → 有延迟和成本代价，先评估是否真的需要
- ❌ 所有优化手段全都加上 → 延迟飙升、性能变差、无法归因
- ❌ 同时改多个变量做优化 → 无法判断哪个起作用
- ❌ top-K 越大越好 → 噪声、Token、延迟和生成干扰都会上升
- ❌ 凭感觉优化，说"效果明显提升" → 必须建立 baseline 和量化指标

面试类

- ❌ 只报技术栈不讲问题 → 每个工具后补"解决了什么问题"
- ❌ 把个人参数配置说成行业标准 → 讲候选范围、对比实验和指标
- ❌ 只讲准确率不讲成本和延迟 → 质量达标后再比延迟和成本
- ❌ 编造指标 → 讲清统计口径和方法区间，不能现场编数据

附录 F：术语表

术语	全称 / 含义
RAG	Retrieval-Augmented Generation，检索增强生成
Chunk / Chunking	文本切片 / 切片策略
Overlap	相邻 chunk 之间的重叠内容
Embedding	文本到高维向量的映射（语义压缩）
Bi-Encoder / 双塔模型	Query 与文档独立编码的 Embedding 模型结构
Cross-Encoder	Query 与文档拼接后联合输入的模型，用于 Rerank
Rerank	对召回候选重新精细打分排序
ANN	Approximate Nearest Neighbor，近似最近邻搜索
HNSW	Hierarchical Navigable Small World，分层可导航小世界图索引
IVF	Inverted File Index，聚类倒排索引
Hybrid Search	混合检索，向量检索 + 关键词检索
BM25	经典关键词相关性打分算法
RRF	Reciprocal Rank Fusion，倒数排名融合
Metadata Filtering	元数据过滤
Parent-Child Retrieval	父子块检索，小检索大使用
Context Compression	上下文压缩
Multi-Query	多角度查询扩展
AST	Abstract Syntax Tree，抽象语法树
ACL	Access Control List，访问控制列表
Recall@K	召回率，正确答案出现在 top-K 的比例
Precision@K	精准率，top-K 中相关结果的比例
MRR	Mean Reciprocal Rank，平均倒数排名
NDCG	Normalized Discounted Cumulative Gain，归一化折损累计增益
RAGAS	RAG Assessment，基于 LLM 的 RAG 生成质量评估框架
Faithfulness	忠实度，答案是否基于检索上下文
Grounded	有根据的，答案有上下文支撑
Agentic RAG	把检索作为 Agent 按需调用工具的 RAG 架构模式

术语	全称 / 含义
ReAct	Reasoning + Acting，推理与行动交替的 Agent 范式
Self-RAG	按需检索 + 自我反思的 RAG 变体
CRAG	Corrective RAG，先评估检索结果、不可靠时纠正
Adaptive-RAG	按问题复杂度选择不检索/单步/多步检索
P95 延迟	95% 请求的延迟上界，衡量尾部延迟

全书收束：一页记住 RAG

```
┌──────────────────────────────────────────────────────────────┐
│  RAG = 检索 + 生成                                           │
│  本质：给大模型装一个可检索、可更新的外置知识库              │
├──────────────────────────────────────────────────────────────┤
│  为什么需要：幻觉 · 私有知识 · 时效性 · 无法溯源（四大局限）  │
├──────────────────────────────────────────────────────────────┤
│  怎么搭（链路）                                              │
│    离线：加载 → 清洗 → 切片(Chunking) → 向量化(Embedding) → 入库 │
│    在线：Query →（改写）→ 召回 top-K →（混合检索/Rerank）→ 拼 Context │
│          → LLM 生成 → 带引用答案                             │
│    铁律：离线在线必须一致；检索要精准不要全面                │
├──────────────────────────────────────────────────────────────┤
│  怎么优化（四侧）                                            │
│    Query 侧：改写 · Multi-Query                              │
│    检索侧：混合检索(RRF) · 元数据过滤                        │
│    精排侧：Rerank（Recall 够好才加）                         │
│    Context 侧：父子块 · Context 压缩                         │
│    原则：先 baseline → 先解决"能不能找到" → 每次只改一个变量 │
├──────────────────────────────────────────────────────────────┤
│  怎么排障                                                    │
│    第一步永远是：手动看 top-K 里有没有正确答案               │
│    有 → 生成侧；没有 → 检索侧；再用 Recall@K 量化            │
├──────────────────────────────────────────────────────────────┤
│  怎么评估                                                    │
│    检索侧：Recall / Precision / MRR / NDCG                   │
│    生成侧：Faithfulness / Answer Relevancy /                 │
│            Context Relevancy / Context Recall（RAGAS）       │
│    线上：采用率（复制率/点赞率/点踩率）才是最终裁判         │
│    闭环：Baseline → 定位瓶颈 → 单变量优化 → 验证 → 持续迭代 │
├──────────────────────────────────────────────────────────────┤
│  怎么答（面试）                                              │
│    五步：结论 → 项目事实 → 方案取舍 → 指标证据 → 适用边界    │
│    讲问题怎么发现、方案为什么选、收益怎么验证                │
└──────────────────────────────────────────────────────────────┘
```

最后一句话：RAG 的每一个设计决策，最终都在回答三个问题——**内容能不能被找到（检索）、找到了能不能被用好（生成）、改完能不能被证明（评估）。**